Deep Learning in 100 Hours

A Four-Week Path from PyTorch to Medical VQA

Jikai

2026

How to Use This Book

This book is a four-week, 20-day, 100-hour path from PyTorch foundations to a reproducible medical visual-question-answering experiment. It assumes a strong mathematical and statistical background but only partial experience building complete PyTorch training systems.

Week	Focus	Hours
1	PyTorch, MLPs, training pipelines, CNNs, and RNNs	25
2	tokenization, attention, Transformer blocks, GPT, and LLaMA components	25
3	LoRA, QLoRA, PEFT engineering, QA metrics, and rank ablations	25
4	ViT, CLIP, VLM fusion, PathVQA, and the capstone	25

Each chapter corresponds to one five-hour learning day and includes concepts, derivations, runnable-code guidance, hands-on tasks, and acceptance questions. Use the separate notebooks in the companion repository for coding practice. The notes explain and organize cited external work; they do not claim originality for the methods.

Study levels

Core Run Complete the lightweight local exercise and pass its behavioral checks.

Light Demo Understand the derivation and run only a small CPU-friendly demonstration.

Understand Only Trace shapes, parameters, memory, and configuration without downloading or training a large model.

Optional Heavy Use external pretrained models, large datasets, GPUs, or full multi-seed experiments only when time and resources permit.

The first month prioritizes conceptual structure and reliable small experiments. It does not require full QLoRA training, large-model reproduction, or a clinically meaningful PathVQA system.

Twenty-day route

Day	Level	Focus
1	Core Run	tensors, shapes, and autograd
2	Core Run	MLPs, activations, and losses
3	Core Run	data pipelines, training, validation, and metrics

Day	Level	Focus
4	Core Run	CNNs, convolution, and image classification
5	Light Demo	RNNs, LSTMs, GRUs, checkpoints, and debugging
6–9	Core Run	tokenization, attention, Transformer blocks, and Tiny GPT
10	Light Demo	LLaMA components
11	Core Run	LoRA from first principles
12	Light Demo	quantization and QLoRA concepts
13	Understand Only	Hugging Face and PEFT engineering map
14	Core Run	QA metrics and split auditing
15–18	Light Demo	LoRA ablation, ViT, CLIP, and VLM fusion
19	Understand + Data Audit	PathVQA data and medical boundaries
20	Core Small / Optional Heavy	lightweight capstone; real PathVQA is optional

Key idea - Recommended daily workflow

Read and annotate the chapter, reproduce the essential derivations, complete only the work required by its study level, answer the acceptance questions from memory, and record results in the experiment log.

The lightweight route is the default. External pretrained models, GPU-heavy dependencies, and restricted datasets are optional and should be used only after checking license, access, memory, and privacy requirements.

Common pitfall

The PathVQA material is for computational education and research-method training. It is not clinically validated and must not be used for diagnosis, treatment, triage, or patient-specific decisions.

Contents

How to Use This Book **ii**

I PyTorch Foundations, CNNs, and RNNs **1**

1 Tensors, Autograd, and Shapes **2**
1.1 Five-hour plan . . . 2
1.2 Tensors as data with structure . . . 2
1.3 Storage, strides, and reshaping . . . 3
1.4 Broadcasting . . . 3
1.5 Matrix multiplication and shape tracing . . . 4
1.6 Dynamic computation graphs . . . 4
1.7 Linear-regression derivation . . . 5
1.8 Stopping gradient tracking . . . 5
1.9 Hands-on work . . . 6
1.10 Acceptance questions . . . 6

2 Multilayer Perceptrons and Loss Functions **7**
2.1 Five-hour plan . . . 7
2.2 From linear models to MLPs . . . 7
2.3 Activations . . . 8
2.4 Logits and softmax . . . 8
2.5 Binary-classification formulations . . . 9
2.6 Parameter counting . . . 9
2.7 Initialization and variance . . . 9
2.8 A complete small classifier . . . 9
2.9 Backpropagation through a two-layer MLP . . . 10
2.10 Accuracy can mislead . . . 11
2.11 Hands-on work . . . 11
2.12 Acceptance questions . . . 11

3 Data Pipelines, Training, Validation, and Metrics **12**
3.1 Five-hour plan . . . 12
3.2 The data contract . . . 12
3.3 Splitting before learning from data . . . 13
3.4 Random, stratified, and grouped splits . . . 13
3.5 Reproducible loading . . . 14
3.6 Collation and variable length . . . 14

3.7 Pipeline assertions . . . 15
3.8 The training step . . . 15
3.9 Optimizers and learning rate . . . 15
3.10 Validation and model mode . . . 16
3.11 Regularization and generalization controls . . . 16
3.12 One-batch overfit test . . . 16
3.13 Hands-on work . . . 17
3.14 Acceptance questions . . . 17

4 Convolutional Neural Networks 18
4.1 Five-hour plan . . . 18
4.2 Why convolution? . . . 18
4.3 Output shapes . . . 19
4.4 What a convolution computes . . . 19
4.5 Stride, padding, pooling, and receptive field . . . 19
4.6 A small PyTorch CNN . . . 20
4.7 CNN versus MLP . . . 20
4.8 Hands-on work . . . 20
4.9 Acceptance questions . . . 21

5 RNNs, LSTMs, GRUs, and Reliable Experiments 22
5.1 Five-hour plan . . . 22
5.2 Vanilla recurrent networks . . . 22
5.3 Backpropagation through time . . . 23
5.4 LSTM . . . 23
5.5 GRU . . . 23
5.6 A lightweight sequence model . . . 23
5.7 RNN versus Transformer . . . 24
5.8 Checkpoints and debugging . . . 24
5.9 Hands-on work . . . 24
5.10 Acceptance questions . . . 25

II Transformer, GPT, and LLaMA 26

6 Tokens, Embeddings, and Next-Token Prediction 27
6.1 Five-hour plan . . . 27
6.2 From text to token IDs . . . 27
6.3 Embedding lookup . . . 28
6.4 Constructing next-token examples . . . 29
6.5 Autoregressive likelihood . . . 29
6.6 Hands-on work . . . 30
6.7 Acceptance questions . . . 30

7 Scaled Dot-Product and Multi-Head Attention 31
7.1 Five-hour plan . . . 31
7.2 Queries, keys, and values . . . 31
7.3 Why divide by $\sqrt{D}$? . . . 32

7.4 Causal masking . . . 32
7.5 Padding masks are a separate constraint . . . 33
7.6 Multi-head shape trace . . . 33
7.7 Parameters and complexity . . . 34
7.8 Tests that expose mask errors . . . 34
7.9 Hands-on work . . . 34
7.10 Acceptance questions . . . 35

8 Transformer Blocks and Stable Training 36
8.1 Five-hour plan . . . 36
8.2 The decoder block . . . 37
8.3 Layer normalization . . . 37
8.4 Residual connections and gradients . . . 37
8.5 The feed-forward network . . . 38
8.6 A complete pre-norm block . . . 38
8.7 Initialization and residual scale . . . 39
8.8 Tests and ablations . . . 39
8.9 Hands-on work . . . 39
8.10 Acceptance questions . . . 39

9 Tiny GPT: Training, Sampling, and Overfit Checks 41
9.1 Five-hour plan . . . 41
9.2 Decoder-only architecture . . . 41
9.3 Training is parallel; generation is sequential . . . 42
9.4 Weight tying . . . 42
9.5 Temperature . . . 43
9.6 Top-k and nucleus sampling . . . 43
9.7 Parameter budget . . . 43
9.8 The tiny-overfit test . . . 44
9.9 A minimal training step . . . 44
9.10 Hands-on work . . . 44
9.11 Acceptance questions . . . 45

10 From GPT to LLaMA: RMSNorm, RoPE, and SwiGLU 46
10.1 Five-hour plan . . . 46
10.2 What changes and what stays . . . 46
10.3 RMSNorm . . . 47
10.4 Rotary position embeddings . . . 48
10.5 SwiGLU . . . 48
10.6 A LLaMA-style block skeleton . . . 49
10.7 Component-level tests . . . 49
10.8 Hands-on work . . . 49
10.9 Weekly acceptance . . . 50

III LoRA, QLoRA, and QA Adaptation 51

11 LoRA: Low-Rank Adaptation 52
11.1 Five-hour plan . . . 52
11.2 The adaptation problem . . . 52
11.3 Parameter count . . . 53
11.4 Initialization and initial equivalence . . . 53
11.5 A teaching implementation . . . 53
11.6 Rank, alpha, and dropout . . . 54
11.7 Choosing target modules . . . 54
11.8 Merging for inference . . . 54
11.9 Required tests . . . 55
11.10 Hands-on work . . . 55
11.11 Acceptance questions . . . 55

12 QLoRA, Quantization Error, and Memory Accounting 56
12.1 Five-hour plan . . . 56
12.2 What QLoRA actually trains . . . 56
12.3 Uniform symmetric fake quantization . . . 57
12.4 NF4 . . . 57
12.5 Double quantization and paged optimizers . . . 58
12.6 Weight-storage floor . . . 58
12.7 The training-memory ledger . . . 58
12.8 How to measure peak memory . . . 58
12.9 Hands-on work . . . 59
12.10 Acceptance questions . . . 59

13 Optional Hugging Face and PEFT Engineering Path 60
13.1 Five-hour plan . . . 60
13.2 From equations to libraries . . . 60
13.3 Preflight before downloading a model . . . 61
13.4 Inspect before injecting adapters . . . 61
13.5 A documented LoRA configuration . . . 61
13.6 A documented 4-bit configuration . . . 62
13.7 Data and collation . . . 62
13.8 Saving a reproducible adapter . . . 63
13.9 Two valid execution routes . . . 63
13.10 Lightweight verification . . . 63
13.11 Notebook audit . . . 63
13.12 Acceptance questions . . . 64

14 Question Answering Data, Metrics, and Split Auditing 65
14.1 Five-hour plan . . . 65
14.2 Three QA formulations . . . 65
14.3 Answer normalization . . . 66
14.4 Exact Match . . . 66
14.5 Token F1 . . . 66
14.6 Baselines and stratification . . . 67

14.7 Split leakage . . . 67
14.8 Failure-case records . . . 68
14.9 Hands-on work . . . 68
14.10 Acceptance questions . . . 68

15 LoRA Rank-Ablation Mini-Project **69**
15.1 Five-hour plan . . . 69
15.2 Research question . . . 69
15.3 Rank and approximation error . . . 70
15.4 Write hypotheses first . . . 70
15.5 Experimental controls . . . 70
15.6 Seeds and fair stopping . . . 71
15.7 Run manifest . . . 71
15.8 Lightweight synthetic task . . . 71
15.9 Hands-on work . . . 72
15.10 Results to record . . . 72
15.11 Selecting a rank . . . 72
15.12 Failure analysis . . . 73
15.13 Weekly acceptance . . . 73

IV ViT, CLIP, VLM, and Medical PathVQA **74**

16 Vision Transformer **75**
16.1 Five-hour plan . . . 75
16.2 Images as token sequences . . . 75
16.3 Convolutional patch projection . . . 76
16.4 Patch size trade-off . . . 76
16.5 Position and pooling . . . 76
16.6 Inductive bias . . . 77
16.7 Minimal classifier . . . 77
16.8 Hands-on work . . . 77
16.9 Acceptance questions . . . 77

17 CLIP and Image-Text Contrastive Learning **79**
17.1 Five-hour plan . . . 79
17.2 Two encoders, one space . . . 79
17.3 Symmetric loss . . . 80
17.4 Retrieval metrics . . . 80
17.5 False negatives and shortcuts . . . 80
17.6 Zero-shot classification . . . 80
17.7 Hands-on work . . . 81
17.8 Acceptance questions . . . 81

18 Vision-Language Fusion and Generation **82**
18.1 Five-hour plan . . . 82
18.2 Three fusion patterns . . . 82
18.3 Closed versus generative answers . . . 83

18.4 A lightweight fusion baseline . . . 83
18.5 Modality-use tests . . . 83
18.6 Freezing and adapters . . . 84
18.7 Hands-on work . . . 84
18.8 Acceptance questions . . . 84

19 PathVQA Data, Metrics, and Medical Boundaries 85
19.1 Five-hour plan . . . 85
19.2 Dataset context . . . 85
19.3 Data card . . . 86
19.4 Leakage and duplication . . . 86
19.5 Evaluation protocol . . . 86
19.6 Notebook audit . . . 86
19.7 Failure taxonomy . . . 87
19.8 Medical-use limitations . . . 87
19.9 Hands-on work . . . 87
19.10 Acceptance questions . . . 87

20 Capstone, Reproduction Package, and Oral Defense 89
20.1 Five-hour plan . . . 89
20.2 Smallest runnable check . . . 89
20.3 Minimum experiment . . . 89
20.4 Pre-registered questions . . . 90
20.5 Required result table . . . 90
20.6 Reproduction package . . . 90
20.7 Evidence ladder . . . 91
20.8 Ten-minute defense . . . 91
20.9 Final acceptance . . . 91

V Reference Appendices 92

A Mathematical Reference 93
A.1 Shape convention . . . 93
A.2 Frequently used derivatives . . . 93
A.3 Probability reminders . . . 93
A.4 Convolution reminders . . . 93
A.5 Recurrent-network reminders . . . 94

B Week 1 Foundations Checklist 95

C Attention Shape and Parameter Reference 96
C.1 Core symbols . . . 96
C.2 End-to-end shape trace . . . 96
C.3 Approximate parameter counts . . . 97
C.4 Mask sanity checks . . . 97

D Week 2 Acceptance Checklist 98

E LoRA and QLoRA Reference **100**
E.1 LoRA shapes . . . 100
E.2 Quick formulas . . . 100
E.3 Configuration manifest . . . 100
E.4 Terminology check . . . 101

F Week 3 Acceptance Checklist **102**

G Multimodal Shape and Evaluation Reference **104**
G.1 Shape table . . . 104
G.2 Core formulas . . . 104
G.3 Minimum baseline ladder . . . 104

H Week 4 and Final Acceptance Checklist **106**

Bibliography **108**

Part I

PyTorch Foundations, CNNs, and RNNs

Chapter 1

Tensors, Autograd, and Shapes

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- interpret a tensor by shape, dtype, device, stride, and semantic axes;
- predict the result of indexing, reshaping, broadcasting, and matrix multiplication;
- explain how PyTorch constructs a dynamic computation graph;
- derive a linear-regression gradient and verify it with autograd;
- distinguish `detach`, `no_grad`, and `inference_mode`.

1.1 Five-hour plan

Time	Activity
60 min	tensors, semantic axes, strides, reshaping, broadcasting, and matrix multiplication
60 min	computation graphs, reverse-mode differentiation, and the linear-regression gradient
120 min	run and extend the tensor/autograd exercise, including manual and numerical checks
30 min	inspect the instructor material or PyTorch autograd documentation and record the training sequence
30 min	acceptance questions and daily experiment log

1.2 Tensors as data with structure

A tensor is not merely a multidimensional array. Its meaning depends on the interpretation assigned to each axis. A tensor of shape $(32, 10)$ could represent 32 observations with 10 features, 32 time steps with 10 channels, or something else. Write the semantic meaning before manipulating the numbers.

We use the conventions in table 1.1.

Table 1.1: Common semantic axes.

Symbol	Meaning	Example
B	batch size	independent observations
T	sequence length	tokens or time steps
C	channel or feature width	embedding dimension
H, W	image height and width	spatial axes
K	class count	output logits

A tensor also has a *dtype* and *device*. The dtype determines representation and precision; the device determines where computation occurs. A common failure is combining a CPU tensor with a CUDA model, or an integer tensor with an operation expecting floating-point gradients.

```
import torch

x = torch.randn(32, 10, dtype=torch.float32)
print(x.shape, x.dtype, x.device)

labels = torch.tensor([0, 1, 1], dtype=torch.long)
```

Class indices passed to `CrossEntropyLoss` normally use `torch.long`; model inputs and parameters normally use floating-point types.

1.3 Storage, strides, and reshaping

Shape describes the visible axes, while stride describes how far the storage pointer moves when an index changes by one. Transposition often creates a non-contiguous view: the same storage is interpreted using different strides.

```
x = torch.arange(12).reshape(3, 4)
y = x.transpose(0, 1)

print(x.shape, x.stride(), x.is_contiguous())
print(y.shape, y.stride(), y.is_contiguous())
```

`view` requires a compatible storage layout. `reshape` returns a view when possible and otherwise copies. Calling `contiguous()` creates storage in the standard row-major order. This distinction matters for both correctness and performance.

Common pitfall

Do not use `reshape` to silence a shape misunderstanding. First state what each axis means and which permutation is required. A program can run with a semantically wrong shape.

1.4 Broadcasting

PyTorch compares dimensions from right to left. Two dimensions are compatible when they are equal or one of them is 1; a missing leading dimension behaves like 1.

For $\boldsymbol{X} \in \mathbb{R}^{B\times C}$ and $\boldsymbol{b} \in \mathbb{R}^{C}$,

$$\boldsymbol{Y} = \boldsymbol{X} + \boldsymbol{b}$$

broadcasts $\boldsymbol{b}$ over the batch axis. This is the intended bias addition.

The classic trap is subtracting a target of shape $(B,)$ from a prediction of shape $(B, 1)$. Right-to-left alignment treats them as $(B, 1)$ and $(1, B)$, producing a (B, B) matrix instead of B residuals.

```
B = 4
prediction = torch.randn(B, 1)
target = torch.randn(B)

wrong = prediction - target       # shape: (B, B)
right = prediction.squeeze(1) - target  # shape: (B,)
```

Checkpoint

Before running a broadcast expression, align the shapes on paper from the right. If the result is not what you intend, insert or remove an axis explicitly.

1.5 Matrix multiplication and shape tracing

For

$$\boldsymbol{X} \in \mathbb{R}^{B\times C}, \qquad \boldsymbol{W} \in \mathbb{R}^{O\times C}, \qquad \boldsymbol{b} \in \mathbb{R}^{O},$$

a PyTorch linear layer computes

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}^{\top} + \boldsymbol{b} \in \mathbb{R}^{B\times O}. \tag{1.1}$$

The weight shape used by `nn.Linear(C,O)` is (O, C), which is why the mathematical expression contains $\boldsymbol{W}^{\top}$.

When an operation fails, write a shape trace:

$$(B, C)@(C, O) \longrightarrow (B, O).$$

When an operation succeeds unexpectedly, shape tracing is even more important because broadcasting may have concealed the error.

1.6 Dynamic computation graphs

If a floating-point tensor has `requires_grad=True`, PyTorch records differentiable operations involving it. The resulting tensors carry a `grad_fn` that points backward through the graph. Calling `backward()` performs reverse-mode automatic differentiation.

Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is a scalar loss. Reverse mode propagates a scalar seed $\partial f/\partial f = 1$ backward and efficiently computes all components of $\nabla_{\boldsymbol{x}} f$. For a non-scalar output $\boldsymbol{y}$, there is no unique implicit seed; `backward` therefore requires an upstream vector, computing a vector-Jacobian product.

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x.square()

# Equivalent to the gradient of y.sum()
y.backward(torch.ones_like(y))
print(x.grad)  # tensor([2., 4.])
```

Gradients accumulate rather than overwrite:

```
x = torch.tensor(2.0, requires_grad=True)
(x ** 2).backward()
print(x.grad)  # 4
(x ** 2).backward()
print(x.grad)  # 8
```

This is why a training step normally calls `optimizer.zero_grad(set_to_none=True)` before the next backward pass.

1.7 Linear-regression derivation

Let $\boldsymbol{X} \in \mathbb{R}^{B\times C}$, $\boldsymbol{w} \in \mathbb{R}^{C}$, and $\boldsymbol{y} \in \mathbb{R}^{B}$. Define

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}, \tag{1.2}$$

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{B}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2. \tag{1.3}$$

Set $\boldsymbol{r} = \boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}$. Using the differential,

$$\begin{aligned} d\mathcal{L} &= \frac{2}{B}\boldsymbol{r}^\top d\boldsymbol{r} \\ &= \frac{2}{B}\boldsymbol{r}^\top \boldsymbol{X}\, d\boldsymbol{w} \\ &= \left[\frac{2}{B}\boldsymbol{X}^\top \boldsymbol{r}\right]^\top d\boldsymbol{w}. \end{aligned}$$

Therefore,

$$\boxed{\nabla_{\boldsymbol{w}}\mathcal{L} = \frac{2}{B}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}).} \tag{1.4}$$

Check the shapes:

$$(C, B)(B,) \longrightarrow (C,),$$

which matches the shape of $\boldsymbol{w}$.

```
B, C = 8, 3
X = torch.randn(B, C)
y = torch.randn(B)
w = torch.randn(C, requires_grad=True)

loss = ((X @ w - y) ** 2).mean()
loss.backward()

manual = (2 / B) * X.T @ (X @ w.detach() - y)
print((w.grad - manual).abs().max())
```

1.8 Stopping gradient tracking

`detach()` Returns a tensor sharing storage with the original but disconnected from its current graph. Use it when a value should be treated as constant downstream.

`torch.no_grad()` Disables graph recording within a context. It is useful for parameter updates or evaluation code that may still manipulate tensors flexibly.

`torch.inference_mode()` A stronger evaluation context that disables additional autograd bookkeeping. It can be faster but is more restrictive.

1.9 Hands-on work

Run `exercises/day01_tensors_autograd.py`. Then:

1. Replace `mean()` with `sum()` and predict the gradient scaling before running.
2. Create the faulty $(B, 1) - (B,)$ broadcast and add an assertion that catches it.
3. Reproduce equation (1.4) with both a loop implementation and a vectorized implementation.
4. Record one example each for `detach`, `no_grad`, and `inference_mode`.

1.10 Acceptance questions

1. Why does a non-scalar output require an upstream gradient for `backward`?
2. Why can a transposed tensor fail with `view`?
3. Why is a successful broadcast not evidence that the program is correct?
4. Explain gradient accumulation and the effect of `set_to_none=True`.
5. Derive equation (1.4) without consulting the text.

Chapter 2

Multilayer Perceptrons and Loss Functions

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

- explain why stacked affine layers require nonlinear activations;
- distinguish logits, probabilities, predictions, and losses;
- derive the softmax cross-entropy gradient;
- count parameters and reason about initialization;
- diagnose underfitting, overfitting, and misleading accuracy.

2.1 Five-hour plan

Time	Activity
50 min	MLPs, activations, logits, losses, and parameter counting
70 min	derive two-layer backpropagation in matrix form and verify every shape
120 min	train the lightweight classifier and compare manual, autograd, and finite-difference gradients
30 min	inspect the instructor material or a primary PyTorch source and record one implementation difference
30 min	acceptance questions and daily experiment log

2.2 From linear models to MLPs

A linear classifier maps $\boldsymbol{x} \in \mathbb{R}^C$ to logits $\boldsymbol{z} \in \mathbb{R}^K$:

$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

Its decision boundaries are hyperplanes. Composing affine maps without a nonlinear activation does not increase expressiveness:

$$\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 = (\boldsymbol{W}_2\boldsymbol{W}_1)\boldsymbol{x} + (\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2).$$

An MLP alternates affine maps and nonlinearities:

$$\boldsymbol{h} = \phi(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1), \qquad \boldsymbol{z} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2. \tag{2.1}$$

The hidden width controls capacity, while the activation prevents collapse to one affine transformation.

2.3 Activations

ReLU. $\mathrm{ReLU}(x) = \max(0, x)$ is inexpensive and has derivative 1 on the positive half-line. A unit can become inactive if its preactivation remains negative.

Tanh. $\tanh(x)$ is smooth and zero-centered, but its derivative approaches zero for large $|x|$. This saturation can slow learning.

GELU. GELU smoothly gates values based on their magnitude and is common in Transformers. For Week 1, the important point is that activation choice changes gradient flow and inductive behavior, not merely output range.

Common pitfall

Do not place a softmax activation inside a classifier that is trained with `CrossEntropyLoss`. The loss expects raw logits and applies a numerically stable log-softmax internally.

2.4 Logits and softmax

For logits $\boldsymbol{z} \in \mathbb{R}^K$,

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}. \tag{2.2}$$

Adding a constant c to every logit leaves the probability unchanged. Numerically stable implementations exploit this by subtracting $\max_j z_j$.

For target class y, cross-entropy is

$$\mathcal{L} = -\log p_y = -z_y + \log \sum_{j=1}^{K} e^{z_j}. \tag{2.3}$$

Differentiating equation (2.3) gives

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - \mathbb{1}[k = y]. \tag{2.4}$$

The gradient raises the target logit and lowers competing logits. It is also bounded, which helps explain the useful optimization behavior of the combined loss.

2.5 Binary-classification formulations

Two standard formulations are equivalent in representational capacity:

1. one logit with binary cross-entropy with logits;
2. two logits with multiclass cross-entropy.

The one-logit form models $P(y = 1 \mid x) = \sigma(z)$. The two-logit form models a two-element softmax. Use the formulation expected by the data and metric pipeline, and keep target dtypes/shapes consistent.

2.6 Parameter counting

For an MLP $C \to H \to K$, including biases,

$$N_{\text{params}} = HC + H + KH + K. \tag{2.5}$$

A $2 \to 16 \to 2$ MLP has

$$16 \cdot 2 + 16 + 2 \cdot 16 + 2 = 82$$

parameters. Parameter counting is a useful sanity check: a model with an unexpected count may have a missing layer, an unintended frozen component, or duplicated parameters.

2.7 Initialization and variance

If independent inputs and weights are zero-centered, the variance of a preactivation roughly scales with fan-in. Xavier initialization balances variance for symmetric activations; He initialization accounts for ReLU discarding approximately half of its inputs. The purpose is not perfect variance preservation but avoiding activations and gradients that immediately vanish or explode [5, 6].

2.8 A complete small classifier

```python
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(2, 16),
    nn.Tanh(),
    nn.Linear(16, 2),
)

features = torch.randn(32, 2)
labels = torch.randint(0, 2, (32,))
logits = model(features)
loss = nn.CrossEntropyLoss()(logits, labels)

loss.backward()
```

The interface is:

$$\text{features } (B, 2) \longrightarrow \text{logits } (B, 2), \qquad \text{labels } (B,).$$

2.9 Backpropagation through a two-layer MLP

For a batch $X \in \mathbb{R}^{B\times C}$, hidden width H, and K classes, write the forward pass as

$$Z_1 = XW_1^\top + \boldsymbol{b}_1, \qquad Z_1 \in \mathbb{R}^{B\times H}, \tag{2.6}$$

$$A_1 = \phi(Z_1), \qquad A_1 \in \mathbb{R}^{B\times H}, \tag{2.7}$$

$$Z_2 = A_1W_2^\top + \boldsymbol{b}_2, \qquad Z_2 \in \mathbb{R}^{B\times K}. \tag{2.8}$$

Here $W_1 \in \mathbb{R}^{H\times C}$ and $W_2 \in \mathbb{R}^{K\times H}$. Let $Y \in \mathbb{R}^{B\times K}$ be one-hot targets and $P = \mathrm{softmax}(Z_2)$. For mean cross-entropy, the output error is

$$\Delta_2 = \frac{P - Y}{B}. \tag{2.9}$$

Reverse-mode differentiation reuses each local derivative while propagating one upstream error signal:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \Delta_2^\top A_1, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_2} = \sum_{b=1}^{B} \Delta_{2,b,:}, \tag{2.10}$$

$$\Delta_1 = (\Delta_2 W_2) \odot \phi'(Z_1), \tag{2.11}$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \Delta_1^\top X, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_1} = \sum_{b=1}^{B} \Delta_{1,b,:}. \tag{2.12}$$

Check, for example, that $(K, B)(B, H) = (K, H)$ matches W_2 and $(H, B)(B, C) = (H, C)$ matches W_1.

Key idea - Backpropagation is structured reuse of the chain rule

Backpropagation is not a different derivative rule. It is an efficient evaluation order for vector–Jacobian products on a computation graph. Each node combines an upstream gradient with its local derivative and passes the result toward the leaves.

For a scalar parameter θ, a centered finite-difference check is

$$g_{\mathrm{fd}}(\theta) = \frac{\mathcal{L}(\theta + \varepsilon) - \mathcal{L}(\theta - \varepsilon)}{2\varepsilon}. \tag{2.13}$$

Use double precision, a small deterministic batch, and several values of ε. Finite differences are a diagnostic, not a training method: very large ε causes truncation error, while very small ε exposes floating-point cancellation.

```
loss = loss_fn(model(features), labels)
loss.backward()

with torch.no_grad():
    index = (0, 0)
    eps = 1e-5
    analytical = model[0].weight.grad[index].item()
    original = model[0].weight[index].item()
    model[0].weight[index] = original + eps
    plus = loss_fn(model(features), labels).item()
```

```
    model[0].weight[index] = original - eps
    minus = loss_fn(model(features), labels).item()
    model[0].weight[index] = original
finite_difference = (plus - minus) / (2 * eps)
```

2.10 Accuracy can mislead

If 95 percent of samples belong to class 0, predicting class 0 always gives 95 percent accuracy. Always compare against a trivial baseline and inspect per-class recall, precision, or a confusion matrix. Training accuracy can also be high while the learned boundary is unstable outside sampled regions.

Key idea - Loss versus metric

The loss is a differentiable training objective. A metric is an evaluation summary chosen for the task. They need not be the same, and a lower loss need not change a thresholded metric immediately.

2.11 Hands-on work

Run `exercises/day02_mlp_loss.py`. Perform controlled changes:

1. remove the activation and record the resulting decision-boundary limitation;
2. sweep hidden width through $H \in \{2, 4, 16, 64\}$;
3. compare ReLU and Tanh using the same seed and training budget;
4. set every training label to zero and explain why accuracy becomes meaningless;
5. inspect gradient norms for both linear layers.
6. compare one manual matrix gradient and one finite-difference gradient with autograd.

Change one variable at a time and write a prediction before each run.

2.12 Acceptance questions

1. Prove that two affine layers without an activation reduce to one affine layer.
2. Derive equation (2.4).
3. Why should `CrossEntropyLoss` receive logits rather than probabilities?
4. When can accuracy be high while the model is useless?
5. Count the parameters of a $10 \to 32 \to 4$ MLP.
6. Derive $\partial \mathcal{L}/\partial W_1$ and $\partial \mathcal{L}/\partial W_2$ for a two-layer MLP and state every shape.

Chapter 3

Data Pipelines, Training, Validation, and Metrics

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

- separate sample definition from batching and loading policy;
- construct reproducible train, validation, and test splits;
- fit preprocessing using training data only;
- recognize entity-level leakage, especially in medical data;
- write one correct training and validation epoch;
- aggregate loss and accuracy across uneven batches;
- design assertions that validate the complete pipeline.

3.1 Five-hour plan

Time	Activity
55 min	datasets, loaders, reproducible splits, preprocessing, and leakage
60 min	training/validation modes, SGD, momentum, AdamW, learning rate, and regularization
120 min	build the complete lightweight training and validation pipeline
35 min	inspect optimizer or data-loading documentation and record one reproducibility risk
30 min	acceptance questions and daily experiment log

3.2 The data contract

A model can only be understood relative to its data contract. For each sample, state:

- input fields, shapes, dtypes, and units;

- target meaning, shape, and valid range;
- sample identity and grouping identity;
- preprocessing and augmentation;
- how missing or invalid values are handled.

In PyTorch, a `Dataset` answers "what is sample i?" while a `DataLoader` answers "how are samples batched, shuffled, and delivered?"

```python
from torch.utils.data import Dataset

class Records(Dataset):
    def __init__(self, features, labels, ids):
        assert len(features) == len(labels) == len(ids)
        self.features = features
        self.labels = labels
        self.ids = ids

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, index):
        return {
            "x": self.features[index],
            "y": self.labels[index],
            "id": self.ids[index],
        }
```

3.3 Splitting before learning from data

Any transformation whose parameters are estimated from data must be fit using the training set only. For standardization,

$$\tilde{x}_j = \frac{x_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}.$$

Computing μ_j and σ_j on the complete dataset allows validation and test distributions to influence training. The same rule applies to vocabularies, feature selection, imputation, class weights, and learned augmentations.

Common pitfall

"I did not use the test labels" is not enough. Using test inputs to fit normalization or select features is still leakage.

3.4 Random, stratified, and grouped splits

Random split. Appropriate when observations are independent and identically distributed and no higher-level entity repeats.

Stratified split. Preserves class proportions. If class prevalence is p, the observed share in a random sample of size n has approximate variance

$$\mathrm{Var}(\hat{p}) = \frac{p(1-p)}{n}.$$

Small validation sets can therefore fluctuate considerably.

Grouped split. Keeps all observations from one entity in the same partition. For pathology data, the grouping unit may be patient, case, slide, or specimen. Image-level random splitting can put near-duplicate crops from one slide into both training and validation.

Key idea - Choose the independence unit

The split unit should match the level at which you expect future generalization. If deployment sees new patients, patient identity must not cross partitions.

3.5 Reproducible loading

Record both the split seed and the actual IDs in each partition. A seed alone may not reproduce a split if dataset order changes.

```
import torch
from torch.utils.data import DataLoader, random_split

generator = torch.Generator().manual_seed(42)
train_set, valid_set = random_split(
    dataset,
    [800, 200],
    generator=generator,
)

train_loader = DataLoader(
    train_set,
    batch_size=32,
    shuffle=True,
    generator=generator,
)
valid_loader = DataLoader(valid_set, batch_size=64)
```

Training is normally shuffled. Validation and test need not be shuffled, because deterministic order simplifies debugging and case-level analysis.

3.6 Collation and variable length

The default collator stacks equal-shaped tensors. Variable-length sequences require padding, packing, or a custom `collate_fn`. The collator should preserve masks and identifiers, not merely return padded values.

```
def collate_batch(samples):
    xs = torch.stack([sample["x"] for sample in samples])
    ys = torch.tensor([sample["y"] for sample in samples])
```

```
    ids = [sample["id"] for sample in samples]
    return {"x": xs, "y": ys, "id": ids}
```

3.7 Pipeline assertions

Useful automated checks include:

- train, validation, and test IDs are pairwise disjoint;
- grouped entity IDs are pairwise disjoint;
- labels fall in $\{0, \dots, K-1\}$;
- no input contains NaN or infinity after preprocessing;
- feature shape and dtype match the model contract;
- class counts and missingness are reported by partition.

3.8 The training step

A conventional training step follows

$$\text{batch} \to \text{forward} \to \text{loss} \to \text{clear gradients} \to \text{backward} \to \text{optimizer step}.$$

```
model.train()
for features, labels in train_loader:
    features = features.to(device)
    labels = labels.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(features)
    loss = loss_fn(logits, labels)
    loss.backward()
    optimizer.step()
```

Gradients accumulate unless they are cleared. Optimizer state is part of the training process: momentum and Adam retain running statistics, so model weights alone cannot resume a run exactly [4].

3.9 Optimizers and learning rate

Plain stochastic gradient descent applies

$$\theta_{t+1} = \theta_t - \eta g_t,$$

where the learning rate η controls update size. A value that is too large can diverge; a value that is too small can make a correct pipeline appear broken. Momentum maintains a running direction,

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t,$$

which can reduce oscillation and accumulate consistent gradients. Adam rescales updates using running first and second moments. AdamW applies weight decay as a separate parameter shrinkage

rather than mixing it into the adaptive gradient. These methods change optimization behavior, not the definition of the loss.

For every run, record optimizer, learning rate, weight decay, batch size, gradient-accumulation steps, scheduler, and total update count. Compare optimizers only under a controlled budget. Warmup and decay schedules become more important for the Transformer experiments, but the first diagnostic remains a short learning-rate sweep on one fixed batch.

3.10 Validation and model mode

Calling `model.eval()` changes modules such as Dropout and BatchNorm. It does not disable autograd. Conversely, `torch.inference_mode()` disables gradient recording but does not change module mode. Correct validation uses both.

```
model.eval()
total_loss, correct, count = 0.0, 0, 0
with torch.inference_mode():
    for features, labels in valid_loader:
        features, labels = features.to(device), labels.to(device)
        logits = model(features)
        loss = loss_fn(logits, labels)
        batch_size = labels.size(0)
        total_loss += loss.item() * batch_size
        correct += (logits.argmax(-1) == labels).sum().item()
        count += batch_size

mean_loss = total_loss / count
accuracy = correct / count
```

Multiplying each batch mean by its sample count matters when the final batch is smaller. Validation chooses hyperparameters and stopping time; the test set is used only after those choices are frozen.

3.11 Regularization and generalization controls

Dropout randomly suppresses activations during training and is disabled by `eval()`. Weight decay discourages large parameter values. Early stopping limits training using a validation rule fixed in advance. Data augmentation changes the effective training distribution and must be disabled or made deterministic for validation. BatchNorm uses batch statistics during training and stored running statistics during evaluation, whereas LayerNorm uses per-example feature statistics and normally behaves the same in both modes.

Regularization cannot repair data leakage, an incorrect target, or a broken metric. First prove that the model can overfit one small batch; then add generalization controls one at a time.

3.12 One-batch overfit test

Repeatedly optimize one fixed small batch before scaling up. Failure to drive its loss very low often reveals mismatched targets, frozen parameters, a detached graph, an absent optimizer step, or an unsuitable learning rate. Passing does not prove generalization, but failing is strong evidence that the local training path needs repair.

3.13 Hands-on work

Run `exercises/day03_dataloader.py` and `exercises/day03_train_eval.py`, then:

1. implement a dictionary-returning Dataset and custom collator;
2. save the split IDs to a human-readable file;
3. deliberately standardize before splitting and describe the leakage path;
4. add a group-overlap assertion;
5. test the final incomplete batch with `drop_last=False`;
6. verify sample-weighted validation loss;
7. make a small model overfit one fixed batch.
8. compare SGD with momentum and AdamW using the same seed and update count.

3.14 Acceptance questions

1. Which transformations must be fit on training data only?
2. When is stratification insufficient?
3. Why can more loader workers make a small workload slower?
4. What should be the grouping entity for a pathology dataset?
5. Why are saved partition IDs more robust than a saved seed alone?
6. Why are `eval()` and `inference_mode()` not interchangeable?
7. How should mean loss be aggregated across unequal batches?
8. What changes when the learning rate is too large or too small?
9. How do Dropout, BatchNorm, and LayerNorm respond to `train()` and `eval()`?

Chapter 4

Convolutional Neural Networks

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

- compute convolution output shapes and parameter counts;
- explain locality, parameter sharing, stride, padding, and receptive field;
- assemble a small image classifier in PyTorch;
- distinguish convolution, pooling, and channel mixing;
- connect CNN inductive bias to the later study of ViT.

4.1 Five-hour plan

Time	Activity
65 min	convolution, kernels, channels, padding, stride, and pooling
55 min	output-shape, parameter-count, and receptive-field calculations
120 min	implement and overfit a small CNN on a tiny local or synthetic dataset
30 min	compare CNN and MLP inductive biases
30 min	acceptance questions and experiment record

4.2 Why convolution?

Flattening an image and applying an MLP discards explicit spatial organization and gives every output unit a separate parameter for every pixel. A convolutional layer instead uses a small kernel at many spatial locations. This gives two important inductive biases:

- **locality:** nearby pixels interact before distant regions;
- **parameter sharing:** the same detector is applied across locations.

Classic CNN systems established this pattern for image recognition long before Vision Transformers [7].

4.3 Output shapes

For input height H, kernel size K, padding P, stride S, and dilation D, the output height is

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} + 1 \right\rfloor. \tag{4.1}$$

The same formula applies to width. For a standard 2D convolution with C_{in} input channels and C_{out} output channels, the weight tensor has shape

$$(C_{\text{out}}, C_{\text{in}}, K_h, K_w),$$

and the parameter count including bias is

$$C_{\text{out}} C_{\text{in}} K_h K_w + C_{\text{out}}. \tag{4.2}$$

It does not depend on image height or width.

Checkpoint

For input $(B, 3, 32, 32)$ and a 3×3 convolution with 16 output channels, padding 1, and stride 1, predict the output shape and parameter count before running code.

4.4 What a convolution computes

Suppressing batch and channel notation, a cross-correlation-style implementation computes

$$y_{i,j} = b + \sum_{u,v} w_{u,v} x_{i+u, j+v}.$$

Deep-learning libraries commonly call this operation convolution even though the kernel is not spatially reversed. Multiple input channels are summed, while multiple output kernels produce multiple feature maps.

Key idea - Spatial mixing and channel mixing

A convolution mixes information across a local spatial neighborhood and across input channels. A 1×1 convolution performs channel mixing at each location without enlarging the spatial neighborhood.

4.5 Stride, padding, pooling, and receptive field

Padding controls border treatment and can preserve resolution. Stride subsamples the output grid. Pooling reduces spatial resolution using a fixed summary such as a maximum or mean. Downsampling reduces computation but can remove small visual evidence.

The receptive field of a unit is the region of the original input that can influence it. Stacking 3×3 stride-one convolutions increases receptive field gradually while inserting nonlinear transformations. Strides and pooling enlarge the effective jump between neighboring units.

Common pitfall

For medical or fine-grained images, aggressive resize, stride, or pooling may remove a small target before the classifier can use it. Always audit preprocessing and intermediate feature-map resolution.

4.6 A small PyTorch CNN

```python
class SmallCNN(nn.Module):
    def __init__(self, classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),              # 32 -> 16
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
        )
        self.head = nn.Linear(32, classes)

    def forward(self, images):
        x = self.features(images)          # (B, 32, 1, 1)
        return self.head(x.flatten(1))     # (B, classes)
```

Use shape assertions or forward hooks while learning. The classifier should output logits; cross-entropy applies the softmax internally.

4.7 CNN versus MLP

An MLP on flattened pixels can represent spatial patterns but does not encode locality or shared detectors. A CNN uses fewer parameters and builds translation-related structure into the architecture. These biases often improve data efficiency, but they are assumptions rather than universal guarantees.

Week 4 will compare this local, shared-kernel bias with ViT's patch tokens and global self-attention. A strided convolution whose kernel and stride equal the patch size can implement ViT patch projection.

4.8 Hands-on work

Use `exercises/day04_cnn.py` and `src/dl100h/cnn.py`, then:

1. verify every feature-map shape by hand and in code;
2. compare manual and PyTorch parameter counts;
3. overfit one small fixed image batch;
4. replace max pooling with average pooling and record the change;
5. halve input resolution and explain what information and compute change.

4.9 Acceptance questions

1. Why does convolution use fewer parameters than a flattened image MLP?
2. Derive the output shape of a strided padded convolution.
3. What is the difference between feature-map channels and spatial axes?
4. How do stride and pooling affect small targets?
5. Which CNN ideas remain relevant when studying ViT?

Chapter 5

RNNs, LSTMs, GRUs, and Reliable Experiments

Study level: Light Demo

Trace the recurrence and gates, then run one small local sequence example. Mastery of recurrent architectures is not required.

Learning objectives

- trace the hidden-state recurrence of an RNN;
- explain backpropagation through time and gradient instability;
- describe how LSTM and GRU gates control information flow;
- compare sequential recurrence with Transformer attention;
- save checkpoints and apply a high-yield debugging procedure.

5.1 Five-hour plan

Time	Activity
55 min	vanilla RNN recurrence and sequence shapes
55 min	backpropagation through time and gradient problems
55 min	LSTM and GRU gates
75 min	run a tiny sequence demo and compare with a Transformer
40 min	checkpoints, one-batch overfitting, and debugging
20 min	acceptance questions and Week 1 reflection

5.2 Vanilla recurrent networks

For input $x_t \in \mathbb{R}^{d_x}$ and hidden state $h_t \in \mathbb{R}^{d_h}$, a simple RNN computes

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \tag{5.1}$$

$$y_t = W_{hy}h_t + b_y. \tag{5.2}$$

The same matrices are reused at every time step. For batch-first input (B, T, d_x), a recurrent layer commonly returns all hidden states (B, T, d_h) and a final state (L, B, d_h), where L includes layer and direction dimensions according to the implementation.

The recurrence provides a compact summary of the past, but time step t depends on $t-1$. Training and inference therefore have a sequential dependency that limits parallelism.

5.3 Backpropagation through time

Unrolling the recurrence creates a depth-T computation graph with shared parameters. A gradient from a late time step contains products of recurrent Jacobians:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}.$$

Repeated products with singular values below one tend to vanish; values above one can explode. Saturating activations can make the problem worse. Gradient clipping limits extreme updates but does not create missing long-range information.

5.4 LSTM

The LSTM introduces a cell state and gates that regulate memory [8]. One common form is

$$\begin{aligned} f_t &= \sigma(W_f[x_t, h_{t-1}] + b_f), & i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i), \\ o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o), & \widetilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The additive cell update offers a more direct gradient path than repeatedly replacing the entire state.

5.5 GRU

The GRU combines memory and hidden state with fewer gates [9]. A typical form uses an update gate z_t, reset gate r_t, candidate state $\widetilde{h}_t$, and interpolation:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t.$$

LSTM and GRU are alternatives, not a strict quality ranking. Choice depends on data, capacity, latency, and implementation constraints.

5.6 A lightweight sequence model

```
class GRUClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, classes):
        super().__init__()
        self.gru = nn.GRU(input_size, hidden_size, batch_first=True)
        self.head = nn.Linear(hidden_size, classes)

```

```
    def forward(self, sequence):
        outputs, final = self.gru(sequence)
        assert outputs.shape[:2] == sequence.shape[:2]
        return self.head(final[-1])
```

Use packed sequences or masks when padding would otherwise affect the final state. Always record whether lengths include special or padded positions.

5.7 RNN versus Transformer

An RNN compresses the past through a recurrent state and has an inherently sequential path of length T. Self-attention gives each position a direct content-dependent route to other allowed positions and permits parallel processing during teacher-forced training, but standard attention uses quadratic score memory in T.

The purpose of this comparison is not to declare RNNs obsolete. Recurrent models remain useful for streaming, compact state, and some low-latency settings. The comparison explains why attention became attractive for large-scale sequence modeling.

5.8 Checkpoints and debugging

A resumable checkpoint should include model, optimizer, step, configuration, and selection metadata. Set seeds and record software, hardware, precision, data version, and split IDs. A seed alone does not guarantee identical execution across devices and library versions.

Before a full run, make the model overfit one fixed batch. If loss does not fall, inspect in this order:

1. one input, target, and prediction;
2. shapes, lengths, dtypes, devices, and finite values;
3. gradients on intended trainable parameters;
4. optimizer ownership and update steps;
5. learning rate and stochastic preprocessing.

Key idea - Week 1 completion

You do not need mastery of recurrent networks. You should be able to trace their state, explain their optimization difficulty, run one small example, and compare their computational path with attention.

5.9 Hands-on work

Use these lightweight files:

- `exercises/day05_rnn.py`;
- `exercises/day05_checkpoint.py`;
- `src/dl100h/recurrent.py`.

1. run one small RNN or GRU classification example on synthetic sequences;
2. print input, output, and final-state shapes;
3. compare sequence lengths 16, 64, and 256 conceptually or empirically;
4. save and reload a checkpoint, then reproduce one prediction;
5. draw the data-to-checkpoint loop and the unrolled recurrence.

5.10 Acceptance questions

1. Why is an RNN sequential over time?
2. Where do vanishing and exploding gradients come from?
3. What role do LSTM or GRU gates play?
4. How do recurrent state and self-attention differ?
5. Which state must be saved to resume training rather than only run inference?

Part II

Transformer, GPT, and LLaMA

Chapter 6

Tokens, Embeddings, and Next-Token Prediction

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- explain why language models operate on token IDs rather than raw strings;
- compare character, word, and subword tokenization;
- trace token and position embeddings with shapes (B, T) and (B, T, C);
- construct shifted inputs and targets for next-token prediction;
- derive sequence negative log-likelihood and perplexity.

6.1 Five-hour plan

Time	Activity
50 min	Tokenization, vocabulary design, special tokens, and encode/decode invariants.
50 min	Embedding lookup, position information, and tensor-shape tracing.
60 min	Derive the autoregressive objective, cross-entropy, and perplexity.
80 min	Complete and extend the tokenization and data-window exercise.
30 min	Read the tokenizer documentation or selected GPT-2 sections and record findings.
30 min	Acceptance questions and experiment log.

6.2 From text to token IDs

A neural network accepts numbers. A tokenizer therefore defines two maps:

$$\text{encode} : \text{text} \longrightarrow (i_1, \ldots, i_T), \qquad \text{decode} : (i_1, \ldots, i_T) \longrightarrow \text{text},$$

where every i_t is an integer in $\{0, \dots, V-1\}$ and V is the vocabulary size.

Character tokenization has a small vocabulary and transparent behavior, but produces long sequences. Word tokenization shortens common text but handles unseen forms poorly. Subword methods such as byte-pair encoding learn frequently reused fragments and balance vocabulary size against sequence length. GPT-2 uses a byte-level BPE-style tokenizer, which also reduces unknown-character failures [11].

Key idea - A tokenizer is part of the model contract

The tokenizer determines the vocabulary, sequence lengths, special-token IDs, and therefore the meaning of the model's input and output rows. Training and evaluation must use exactly the same tokenizer and vocabulary mapping.

Useful invariants include

$$0 \le i_t < V, \qquad \text{decode}(\text{encode}(s)) \approx s,$$

where the second relation may differ in whitespace normalization for some tokenizers. Common special tokens are padding, beginning-of-sequence, end-of-sequence, and unknown. Do not assume that every model uses all four.

6.3 Embedding lookup

Let $E \in \mathbb{R}^{V \times C}$ be a learned token-embedding table. Looking up token ID i_t selects row $E_{i_t,:}$. For a batch of IDs

$$X_{\text{id}} \in \{0, \dots, V-1\}^{B \times T},$$

the lookup produces

$$X_{\text{tok}} = E[X_{\text{id}}] \in \mathbb{R}^{B \times T \times C}.$$

This is not a dense multiplication in code, although it is equivalent to multiplying a one-hot vector by E. Only rows used in the batch receive nonzero gradients.

Token identity alone does not reveal order. A simple GPT implementation adds a learned position table $P \in \mathbb{R}^{T_{\max} \times C}$:

$$X^{(0)}_{b,t,:} = E_{x_{b,t},:} + P_{t,:}.$$

Later we will replace additive positions with rotary position embeddings.

```python
import torch
from torch import nn

B, T, V, C = 4, 16, 100, 32
token_ids = torch.randint(0, V, (B, T))

token_emb = nn.Embedding(V, C)
position_emb = nn.Embedding(T, C)
positions = torch.arange(T)

x = token_emb(token_ids) + position_emb(positions)[None, :, :]
assert x.shape == (B, T, C)
```

6.4 Constructing next-token examples

Given a token sequence $(s_0, \dots, s_T)$, the input and target are shifted by one position:

$$x = (s_0, \dots, s_{T-1}), \qquad y = (s_1, \dots, s_T).$$

The model sees $x_{\le t}$ and predicts $y_t = x_{t+1}$. A long corpus can be sampled with windows of length $T + 1$.

```
def sample_batch(data, batch_size, context, generator=None):
    starts = torch.randint(
        0, len(data) - context, (batch_size,), generator=generator
    )
    chunks = torch.stack([data[i : i + context + 1] for i in starts])
    x = chunks[:, :-1]
    y = chunks[:, 1:]
    assert x.shape == y.shape == (batch_size, context)
    assert torch.equal(x[:, 1:], y[:, :-1])
    return x, y
```

For padded batches, define a Boolean validity mask $M \in \{0, 1\}^{B \times T}$. Padding positions must contribute neither to the loss nor to reported token accuracy. In PyTorch, one common choice is an ignored target ID.

Common pitfall

A causal mask and a padding mask solve different problems. The causal mask prevents attention to future positions. The padding mask prevents real tokens from attending to artificial padding and excludes padding targets from the loss.

6.5 Autoregressive likelihood

The chain rule factors the probability of a sequence as

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}). \tag{6.1}$$

For logits $Z \in \mathbb{R}^{B \times T \times V}$, the model distribution is

$$p_{b,t,v} = \frac{\exp Z_{b,t,v}}{\sum_{j=1}^{V} \exp Z_{b,t,j}}.$$

The mean token negative log-likelihood over valid positions is

$$\mathcal{L} = -\frac{1}{N} \sum_{b,t} M_{b,t} \log p_{b,t,y_{b,t}}, \qquad N = \sum_{b,t} M_{b,t}. \tag{6.2}$$

Flattening the first two axes gives the input shape expected by cross-entropy:

```
logits = model(x)                  # (B, T, V)
loss = nn.functional.cross_entropy(
    logits.reshape(-1, logits.size(-1)),
    y.reshape(-1),
    ignore_index=pad_id,
)
```

If $\mathcal{L}$ is the average natural-log loss per token, perplexity is

$$\mathrm{PPL} = \exp(\mathcal{L}). \tag{6.3}$$

It can be read as an effective branching factor, but only cautiously. Perplexities are not directly comparable when tokenization, preprocessing, or evaluation data differ.

6.6 Hands-on work

Use `exercises/day06_tokenization.py`. The companion Week 2 notebook is optional reinforcement.

1. Implement character-level `encode` and `decode`; test the round trip on punctuation and newlines.
2. Create x/y windows and assert that they differ by exactly one position.
3. Compare character, whitespace-word, and simple bigram tokenization by vocabulary size and average tokens per sentence.
4. Add padding to a mixed-length batch and prove that changing padded targets does not alter the masked loss.
5. Verify numerically that `torch.exp(loss)` equals the reported perplexity.

6.7 Acceptance questions

1. Why does a larger vocabulary usually shorten sequences but enlarge the embedding and output matrices?
2. What exact relationship must hold between an input window and its target window?
3. Why must training, validation, and deployment share the tokenizer?
4. Why is perplexity incomparable across arbitrary tokenizers?
5. For token IDs of shape (B, T), state the shapes after token lookup, position addition, and vocabulary projection.

Chapter 7

Scaled Dot-Product and Multi-Head Attention

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- derive scaled dot-product attention and explain the scale $1/\sqrt{D}$;
- construct and test a causal mask;
- label every reshape in multi-head self-attention;
- distinguish attention weights from value aggregation;
- state the time and memory complexity in sequence length.

7.1 Five-hour plan

Time	Activity
65 min	Queries, keys, values, similarity scores, and causal masking.
75 min	Derive scaling and trace all single-head and multi-head shapes.
105 min	Implement attention and invariance tests from scratch.
30 min	Read the attention section of *Attention Is All You Need*.
25 min	Acceptance questions and error analysis.

7.2 Queries, keys, and values

For an input $X \in \mathbb{R}^{B\times T\times C}$, learned projections create

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K, W_V \in \mathbb{R}^{C\times C}$ in the usual full-width implementation. A query describes what a position is looking for, a key describes what a position offers for matching, and a value carries the information that will be combined.

For one head of width D, the score between query position i and key position j is

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{D}}.$$

Row-wise softmax gives attention weights a_{ij}, and the output is a weighted sum of values:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{D}} + M\right)V. \tag{7.1}$$

Key idea - Matching and transport are separate

$QK^\top$ decides how strongly positions relate. Multiplication by V transports and mixes content. Attention weights are not themselves the final representation.

7.3 Why divide by $\sqrt{D}$?

Assume independent query and key components with mean zero and variance one. Then

$$q^\top k = \sum_{r=1}^{D} q_r k_r.$$

Each product has variance approximately one, so

$$\mathrm{Var}(q^\top k) \approx D.$$

Dividing by $\sqrt{D}$ restores variance near one. Without scaling, increasing the head width makes logits larger, softmax distributions more saturated, and gradients less informative [10].

7.4 Causal masking

A decoder must not access future targets. Define

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

After adding M before softmax, every forbidden probability is zero. With zero-based indexing, row 0 may attend only to column 0; the last row may attend to all positions.

```
def causal_attention(q, k, v):
    # q, k, v: (B, H, T, D)
    d = q.size(-1)
    scores = q @ k.transpose(-2, -1) / (d ** 0.5)
    t = q.size(-2)
    allowed = torch.ones(t, t, dtype=torch.bool, device=q.device).tril()
    scores = scores.masked_fill(~allowed, float("-inf"))
    weights = scores.softmax(dim=-1)
    return weights @ v, weights
```

The mask must be applied to scores before softmax. Multiplying probabilities by a triangular matrix afterward destroys row normalization unless the rows are renormalized, and it still represents the wrong computation.

7.5 Padding masks are a separate constraint

For padded batches, let $P_{b,j} = 1$ when key position j is real and zero when it is padding. The allowed key set combines both rules:

$$A_{b,i,j} = \mathbf{1}[j \le i]\, P_{b,j}.$$

Scores with $A_{b,i,j} = 0$ are masked before softmax. A causal mask is shared across the batch and blocks future positions; a padding mask is example-specific and blocks artificial tokens. The loss mask is separate again: it determines which target positions contribute to the objective.

Common pitfall

Masking padded keys does not automatically remove padded query outputs from the loss. Carry the validity mask through attention, pooling, and the training objective.

7.6 Multi-head shape trace

Let H divide C and let $D = C/H$. The critical transformations are

$$\begin{aligned} X &: (B, T, C), \\ Q, K, V &: (B, T, C), \\ \text{reshape} &: (B, T, H, D), \\ \text{transpose} &: (B, H, T, D), \\ QK^\top &: (B, H, T, T), \\ AV &: (B, H, T, D), \\ \text{transpose} + \text{reshape} &: (B, T, C), \\ \text{output projection} &: (B, T, C). \end{aligned}$$

Different heads can learn different similarity structures. Concatenation restores the channel width, and the output projection mixes information across heads.

```
class CausalSelfAttention(nn.Module):
    def __init__(self, width, heads):
        super().__init__()
        assert width % heads == 0
        self.heads = heads
        self.head_dim = width // heads
        self.qkv = nn.Linear(width, 3 * width, bias=False)
        self.out = nn.Linear(width, width, bias=False)

    def forward(self, x):
        b, t, c = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)

```

```
        def split_heads(z):
            return z.view(b, t, self.heads, self.head_dim).transpose(1, 2)

        q, k, v = map(split_heads, (q, k, v))
        y, weights = causal_attention(q, k, v)
        y = y.transpose(1, 2).contiguous().view(b, t, c)
        return self.out(y), weights
```

The call to `contiguous()` is deliberate: transposing changes strides, so `view` may otherwise fail or misrepresent the intended layout.

7.7 Parameters and complexity

Ignoring biases, the Q, K, V, and output projections contain approximately

$$3C^2 + C^2 = 4C^2$$

parameters, independent of T. However, the score tensor has BHT^2 elements. Standard attention therefore uses $O(BT^2C)$ compute and $O(BHT^2)$ score memory. This quadratic dependence is the central long-context bottleneck.

7.8 Tests that expose mask errors

Shape tests are necessary but insufficient. Use behavioral tests:

1. Change only future tokens and prove that earlier outputs remain unchanged.
2. Check that all weights above the diagonal are exactly zero within tolerance.
3. Check that every attention row sums to one.
4. Compare a vectorized implementation with a slow loop implementation.
5. Remove scaling and compare attention entropy as D grows.

```
model.eval()
x2 = x.clone()
x2[:, 5:, :] = torch.randn_like(x2[:, 5:, :])
with torch.no_grad():
    y1, _ = attn(x)
    y2, _ = attn(x2)
assert torch.allclose(y1[:, :5], y2[:, :5], atol=1e-6)
```

7.9 Hands-on work

Use `exercises/day07_attention.py` and `src/dl100h/attention.py`.

1. Write single-head causal attention in at most 25 lines.
2. Add shape assertions after every transformation.
3. Reverse the mask, run the future-invariance test, and explain the failure.

4. Measure score-tensor memory for $T \in \{64, 128, 256, 512\}$ and verify quadratic growth.
5. Plot one head's attention matrix, but describe it as a diagnostic rather than a guaranteed explanation.
6. Create a mixed-length padded batch and verify that padded keys receive zero attention while valid rows still sum to one.

7.10 Acceptance questions

1. Why is the scale $1/\sqrt{D}$ used instead of $1/D$?
2. For input (B, T, C) with H heads, state every intermediate shape through the output projection.
3. Why must the causal mask be added before softmax?
4. How do attention compute and score memory scale with T?
5. What test proves that token 0 cannot depend on tokens 1 through $T - 1$?
6. Why are causal masks, padding-key masks, and loss masks three different objects?

Chapter 8

Transformer Blocks and Stable Training

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- assemble masked self-attention, an MLP, residual paths, and normalization;
- distinguish token mixing from channel mixing;
- explain pre-norm and post-norm Transformer blocks;
- derive the residual Jacobian and its identity path;
- estimate the parameters and activation cost of one block.

8.1 Five-hour plan

Time	Activity
60 min	Layer normalization, residual connections, and feed-forward networks.
60 min	Pre-norm versus post-norm and residual-gradient derivation.
120 min	Implement a complete decoder block and its tests.
30 min	Parameter counting and memory estimates.
30 min	Acceptance questions and experiment record.

8.2 The decoder block

A Transformer decoder block contains two main transformations: causal self-attention and a position-wise feed-forward network. In a pre-norm block,

$$X' = X + \text{Dropout}(\text{Attn}(\text{LN}(X))), \tag{8.1}$$
$$Y = X' + \text{Dropout}(\text{MLP}(\text{LN}(X'))). \tag{8.2}$$

All tensors retain shape (B, T, C).

Attention mixes information across the T token positions. The MLP is applied independently at each position, so it mixes channels but not tokens. Both are needed: attention routes contextual information, and the MLP transforms the representation at each token.

Key idea - Two axes, two roles

Attention primarily mixes along the sequence axis. The feed-forward network primarily mixes along the channel axis. Shape preservation allows both transformations to be wrapped by residual additions.

8.3 Layer normalization

For a token vector $x \in \mathbb{R}^C$, LayerNorm computes

$$\mu = \frac{1}{C}\sum_{i=1}^{C} x_i, \qquad \sigma^2 = \frac{1}{C}\sum_{i=1}^{C}(x_i - \mu)^2,$$

then

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i. \tag{8.3}$$

Statistics are computed over channels separately for each batch item and token. They are not batch-level statistics, so LayerNorm behaves the same in training and evaluation modes.

Common pitfall

Do not normalize over the sequence axis by accident. For a tensor (B, T, C), standard LayerNorm with `normalized_shape=C` normalizes the final channel axis.

8.4 Residual connections and gradients

For a residual transformation

$$y = x + F(x),$$

the Jacobian is

$$\frac{\partial y}{\partial x} = I + J_F(x). \tag{8.4}$$

If a downstream scalar loss has gradient g_y, then

$$g_x = g_y(I + J_F(x)).$$

The identity term provides a direct path that does not require multiplication by every learned Jacobian. This does not guarantee perfect optimization, but it substantially improves gradient flow in deep networks.

Post-norm applies normalization after the residual sum:

$$Y = \mathrm{LN}(X + F(X)).$$

Pre-norm instead computes

$$Y = X + F(\mathrm{LN}(X)).$$

Modern decoder-only models commonly use pre-norm variants because the residual stream preserves a clean identity route across depth.

8.5 The feed-forward network

The original Transformer uses two linear maps with an activation:

$$\mathrm{MLP}(x) = W_2\phi(W_1x + b_1) + b_2, \tag{8.5}$$

where $W_1 \in \mathbb{R}^{C\times rC}$, $W_2 \in \mathbb{R}^{rC\times C}$, and often $r = 4$. Common activations include ReLU and GELU. The same MLP parameters are reused at every sequence position.

Ignoring biases, the attention projections use about $4C^2$ parameters, while an expansion ratio r gives the MLP about

$$C(rC) + (rC)C = 2rC^2$$

parameters. At $r = 4$, this is $8C^2$, twice the projection count of attention. Parameter location and runtime cost are different: attention scores still scale quadratically with T.

8.6 A complete pre-norm block

```python
class TransformerBlock(nn.Module):
    def __init__(self, width, heads, expansion=4, dropout=0.0):
        super().__init__()
        self.norm1 = nn.LayerNorm(width)
        self.attn = CausalSelfAttention(width, heads)
        self.norm2 = nn.LayerNorm(width)
        hidden = expansion * width
        self.mlp = nn.Sequential(
            nn.Linear(width, hidden),
            nn.GELU(),
            nn.Linear(hidden, width),
            nn.Dropout(dropout),
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        attn_out, weights = self.attn(self.norm1(x))
        x = x + self.dropout(attn_out)
        x = x + self.mlp(self.norm2(x))
        return x, weights
```

The block should accept and return (B, T, C). Test this property for several batch and sequence lengths, including $B = 1$ and $T = 1$.

8.7 Initialization and residual scale

Deep residual stacks repeatedly add updates to the residual stream. Poor initialization can cause activations or updates to grow with depth. A practical small-model baseline is to use framework defaults and monitor activation and gradient statistics. Larger architectures often scale selected residual projections according to depth; the exact convention is architecture-specific and must match the implementation being reproduced.

Record for each block:

- residual-stream mean and standard deviation;
- gradient norm of attention and MLP projections;
- fraction of exactly zero activations, if relevant;
- training loss and validation loss.

These diagnostics are more useful than inspecting loss alone when a deep stack becomes unstable.

8.8 Tests and ablations

1. **Identity test:** zero the final projection of each branch; the block output should equal its input.
2. **Causality test:** modifying future tokens must not change earlier outputs.
3. **Mode test:** dropout should make training outputs stochastic and evaluation outputs deterministic.
4. **Gradient test:** every intended trainable parameter should receive a finite gradient.
5. **Parameter test:** compare manual counts with PyTorch's reported parameter total.

8.9 Hands-on work

Use `exercises/day08_transformer_block.py`. The companion Week 2 notebook is optional reinforcement.

1. Implement pre-norm and post-norm variants behind one configuration flag.
2. Verify the block's input/output shape for several (B, T, C, H) settings.
3. Compute attention, MLP, normalization, and total parameter counts by hand.
4. Train stacks of 2, 4, and 8 blocks on the same tiny batch and compare gradient norms.
5. Explain any observed difference without claiming that one small run proves universal superiority.

8.10 Acceptance questions

1. Which component mixes tokens, and which component mixes channels?
2. Over which axis does LayerNorm compute statistics for (B, T, C)?

3. Derive the residual Jacobian $I + J_F$ and explain the identity path.
4. Why can an MLP contain more parameters while attention consumes more memory for long sequences?
5. How would you make a Transformer block behave exactly like the identity map for a test?

Chapter 9

Tiny GPT: Training, Sampling, and Overfit Checks

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- assemble a decoder-only language model from embeddings and Transformer blocks;
- explain parallel teacher-forced training and autoregressive generation;
- implement temperature, top-k, and nucleus sampling;
- understand output-head weight tying;
- use tiny-batch overfitting as an end-to-end correctness test.

9.1 Five-hour plan

Time	Activity
45 min	GPT architecture and teacher-forced objective.
45 min	Parameter counting, weight tying, and sampling derivations.
150 min	Build, train, debug, and sample from Tiny GPT.
30 min	Compare the implementation with GPT-2 documentation or code.
30 min	Acceptance questions and experiment record.

9.2 Decoder-only architecture

A GPT-style model contains:

1. token embeddings and position information;
2. a stack of causal Transformer blocks;

3. a final normalization layer;

4. a linear language-model head that maps width C to vocabulary size V.

For input IDs (B, T), the final logits have shape (B, T, V). Every position predicts the next token, so a single forward pass supplies BT supervised predictions.

```
class TinyGPT(nn.Module):
    def __init__(self, vocab, context, width, heads, layers):
        super().__init__()
        self.context = context
        self.token_emb = nn.Embedding(vocab, width)
        self.pos_emb = nn.Embedding(context, width)
        self.blocks = nn.ModuleList([
            TransformerBlock(width, heads) for _ in range(layers)
        ])
        self.norm = nn.LayerNorm(width)
        self.lm_head = nn.Linear(width, vocab, bias=False)

    def forward(self, ids):
        b, t = ids.shape
        if t > self.context:
            raise ValueError("sequence exceeds context length")
        pos = torch.arange(t, device=ids.device)
        x = self.token_emb(ids) + self.pos_emb(pos)[None, :, :]
        for block in self.blocks:
            x, _ = block(x)
        return self.lm_head(self.norm(x))
```

9.3 Training is parallel; generation is sequential

During training, the complete shifted input is known. The causal mask ensures that position t cannot use future tokens, so all time positions can be processed in parallel. This is teacher forcing: the context comes from the real training sequence.

During generation, the next token is unknown. The model predicts one distribution, selects a token, appends it, and repeats:

$$x_{1:t} \longrightarrow p(x_{t+1} \mid x_{1:t}) \longrightarrow \hat{x}_{t+1} \longrightarrow x_{1:t+1}.$$

Generation is therefore autoregressive. A key-value cache avoids recomputing keys and values for all previous tokens at every step, but a minimal educational implementation may recompute the context for clarity.

9.4 Weight tying

The token embedding matrix has shape (V, C) and the output-head weight also has shape (V, C). Setting

$$W_{\text{head}} = E_{\text{token}}$$

shares the parameters and removes approximately VC independent weights. In PyTorch:

```
self.lm_head.weight = self.token_emb.weight
```

Weight tying changes both parameter count and gradient flow: the shared matrix learns from its role as input representation and output classifier.

9.5 Temperature

Given logits $z \in \mathbb{R}^V$ and temperature $\tau > 0$,

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \tag{9.1}$$

As $\tau \to 0^+$, probability concentrates on the maximum-logit token. As $\tau \to \infty$, all scaled logits approach zero and the distribution approaches uniform. Temperature changes sampling behavior; it does not retrain the model.

Greedy decoding selects $\arg\max_i z_i$. Sampling draws from the categorical distribution. Greedy decoding is deterministic but can become repetitive; unrestricted sampling can select implausible tail tokens.

9.6 Top-k and nucleus sampling

Top-k sampling retains only the k highest logits and sets all others to $-\infty$. Nucleus, or top-p, sampling sorts tokens by probability and retains the smallest prefix whose cumulative probability reaches a threshold p. Top-k fixes the candidate count; top-p adapts the count to distribution sharpness.

```
@torch.no_grad()
def generate(model, ids, steps, temperature=1.0, top_k=None):
    model.eval()
    for _ in range(steps):
        context = ids[:, -model.context:]
        logits = model(context)[:, -1, :] / temperature
        if top_k is not None:
            values, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            cutoff = values[:, [-1]]
            logits = logits.masked_fill(logits < cutoff, float("-inf"))
        probs = logits.softmax(dim=-1)
        next_id = torch.multinomial(probs, num_samples=1)
        ids = torch.cat([ids, next_id], dim=1)
    return ids
```

Use a strictly positive temperature, check for finite logits, and set a random seed when comparing configurations.

9.7 Parameter budget

For L blocks and expansion ratio r, a rough non-embedding count is

$$L(4C^2 + 2rC^2) + O(LC).$$

Token embeddings contribute VC, learned position embeddings contribute $T_{\max}C$, and an untied output head contributes another VC. This approximation is useful for finding an accidental extra projection or untied head.

Checkpoint

For $V = 5{,}000$, $C = 256$, $L = 6$, and $r = 4$, estimate the embedding, attention, and MLP parameter totals before asking PyTorch. State whether the output head is tied.

9.8 The tiny-overfit test

A language model that cannot memorize one very small batch is probably incorrect, underpowered, or poorly optimized. Use this sequence:

1. select one fixed batch and disable data resampling;
2. disable dropout;
3. train for enough steps with a moderate learning rate;
4. verify that loss falls far below the random baseline $\log V$;
5. decode samples from prefixes present in the batch;
6. restore the normal data pipeline only after the test passes.

If loss stays near $\log V$, inspect target shifting, the causal mask, optimizer updates, logits/target flattening, and padding exclusion. If loss decreases but generation is nonsense, inspect the tokenizer round trip and generation indexing.

9.9 A minimal training step

```
model.train()
optimizer.zero_grad(set_to_none=True)
logits = model(x)                          # (B, T, V)
loss = nn.functional.cross_entropy(
    logits.reshape(-1, logits.size(-1)),
    y.reshape(-1),
)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

Gradient clipping limits the global norm for an update; it does not repair nonfinite activations or an incorrect loss. Log unclipped gradient norm when diagnosing instability.

9.10 Hands-on work

Use `exercises/day09_tiny_gpt.py`. The companion Week 2 notebook is optional reinforcement.

1. Make the model memorize one fixed batch and save the loss curve.
2. Compare manual and reported parameter counts with and without weight tying.
3. Generate from the same prefix using greedy, $\tau = 0.7$, $\tau = 1.2$, and top-k sampling.
4. Demonstrate that training position t is unchanged when future input tokens are perturbed.

5. Record architecture, seed, context length, final loss, and one decoded sample.

9.11 Acceptance questions

1. Why can training process all positions in parallel while generation cannot?
2. What happens to softmax as temperature approaches zero and infinity?
3. How many parameters does weight tying remove?
4. Why is $\log V$ a useful random-prediction loss reference?
5. List five checks to make when a Tiny GPT cannot overfit one batch.

Chapter 10

From GPT to LLaMA: RMSNorm, RoPE, and SwiGLU

Study level: Light Demo

Understand RMSNorm, RoPE, and SwiGLU and run component-level demonstrations. Training a LLaMA model is not required.

Learning objectives

By the end of this chapter, you should be able to:

- identify which decoder-only Transformer ideas GPT and LLaMA share;
- derive RMSNorm and contrast it with LayerNorm;
- explain RoPE as rotations applied to query and key pairs;
- implement a SwiGLU feed-forward network;
- avoid treating an architecture family as one fixed model configuration.

10.1 Five-hour plan

Time	Activity
65 min	GPT/LLaMA comparison, RMSNorm, RoPE, and gated feed-forward layers.
70 min	Derive RMS normalization and the relative-position property of RoPE.
105 min	Implement and test RMSNorm, RoPE, and SwiGLU.
35 min	Read selected LLaMA paper sections and complete the paper template.
25 min	Weekly acceptance and reflection.

10.2 What changes and what stays

LLaMA is still an autoregressive decoder-only Transformer: token representations pass through causal self-attention and feed-forward layers, and the output predicts the next token. The original

LLaMA design changes several internal components, including pre-normalization with RMSNorm, rotary position embeddings, and a SwiGLU feed-forward network [13].

Component	Simple GPT-style baseline	LLaMA-style choice
Normalization	LayerNorm, often pre-norm	RMSNorm, pre-norm
Position	learned additive embeddings	rotary position embeddings on Q, K
Feed-forward	GELU MLP with two projections	SwiGLU with three projections
Attention	causal multi-head attention	causal attention; head sharing varies by release
Objective	next-token prediction	next-token prediction

Exact head counts, context lengths, tokenizer details, attention variants, and dimensions vary across model sizes and LLaMA generations. Always identify the specific model and configuration being discussed.

10.3 RMSNorm

For $x \in \mathbb{R}^C$, define

$$\operatorname{rms}(x) = \sqrt{\frac{1}{C}\sum_{i=1}^{C} x_i^2 + \epsilon}.$$

RMSNorm computes

$$\operatorname{RMSNorm}(x)_i = g_i \frac{x_i}{\operatorname{rms}(x)}, \tag{10.1}$$

with learned scale $g \in \mathbb{R}^C$ [14]. Unlike LayerNorm, it does not subtract the mean and typically has no learned bias.

If x is multiplied by a positive scalar a and ϵ is negligible,

$$\frac{ax}{\sqrt{\frac{1}{C}\sum_i (ax_i)^2}} = \frac{x}{\sqrt{\frac{1}{C}\sum_i x_i^2}},$$

so the normalized direction is invariant to positive rescaling. RMSNorm controls magnitude without recentering the vector.

```python
class RMSNorm(nn.Module):
    def __init__(self, width, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(width))
        self.eps = eps

    def forward(self, x):
        # Compute the statistic in float32 for mixed-precision stability.
        rms_inv = torch.rsqrt(x.float().pow(2).mean(dim=-1, keepdim=True)
                              + self.eps)
        return (x.float() * rms_inv).to(x.dtype) * self.weight
```

Test the module against the equation, verify the output shape, and check approximate scale invariance. Do not expect the output mean to be zero.

10.4 Rotary position embeddings

RoPE rotates pairs of query and key coordinates by a position-dependent angle [15]. For one two-dimensional pair and position m,

$$R(m\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}.$$

The rotated query and key are

$$\widetilde{q}_m = R(m\theta)q_m, \qquad \widetilde{k}_n = R(n\theta)k_n.$$

Because a rotation matrix is orthogonal and $R(a)^\top R(b) = R(b-a)$,

$$\widetilde{q}_m^\top \widetilde{k}_n = q_m^\top R(m\theta)^\top R(n\theta)k_n \tag{10.2}$$

$$= q_m^\top R((n-m)\theta)k_n. \tag{10.3}$$

The dot product therefore contains the relative displacement $n-m$. Multiple coordinate pairs use different frequencies, allowing the representation to encode position at several scales.

Key idea - RoPE changes matching, not values

Rotations are applied to queries and keys before their dot product. Values are not rotated in the standard construction. RoPE modifies how positions match while preserving each pair's Euclidean norm.

One explicit implementation for paired coordinates is:

```
def apply_rope(x, cos, sin):
    # x: (B, H, T, D), D even
    x_even = x[..., 0::2]
    x_odd = x[..., 1::2]
    y_even = x_even * cos - x_odd * sin
    y_odd = x_even * sin + x_odd * cos
    return torch.stack((y_even, y_odd), dim=-1).flatten(-2)
```

The shapes of `cos` and `sin` should broadcast to $(B, H, T, D/2)$. Cache them by position and frequency rather than rebuilding them inside every layer and step.

10.5 SwiGLU

The SiLU, or swish, activation is

$$\mathrm{SiLU}(z) = z\,\sigma(z).$$

A SwiGLU feed-forward layer uses a gate and a value branch:

$$\mathrm{SwiGLU}(x) = W_{\mathrm{down}}\left(\mathrm{SiLU}(W_{\mathrm{gate}}x) \odot (W_{\mathrm{up}}x)\right). \tag{10.4}$$

The elementwise product lets one branch modulate the other [16].

If the intermediate width is M, the three bias-free matrices contain approximately

$$CM + CM + MC = 3CM$$

parameters. To compare fairly with a two-matrix MLP of width $4C$ and approximately $8C^2$ parameters, choose M near $8C/3$, often rounded for hardware efficiency.

```python
class SwiGLU(nn.Module):
    def __init__(self, width, hidden):
        super().__init__()
        self.gate = nn.Linear(width, hidden, bias=False)
        self.up = nn.Linear(width, hidden, bias=False)
        self.down = nn.Linear(hidden, width, bias=False)

    def forward(self, x):
        return self.down(nn.functional.silu(self.gate(x)) * self.up(x))
```

10.6 A LLaMA-style block skeleton

```python
class LlamaStyleBlock(nn.Module):
    def __init__(self, width, heads, hidden):
        super().__init__()
        self.attn_norm = RMSNorm(width)
        self.attn = RotaryCausalAttention(width, heads)
        self.ffn_norm = RMSNorm(width)
        self.ffn = SwiGLU(width, hidden)

    def forward(self, x, cos, sin):
        x = x + self.attn(self.attn_norm(x), cos, sin)
        x = x + self.ffn(self.ffn_norm(x))
        return x
```

The skeleton deliberately leaves the attention implementation as an exercise. Insert RoPE after splitting Q and K into heads and before computing $QK^\top$.

10.7 Component-level tests

1. **RMSNorm equation:** compare the module to a direct formula on random input.
2. **Norm preservation:** verify that RoPE preserves each rotated pair's squared norm.
3. **Relative displacement:** compare two pairs with the same $n - m$ at different absolute positions.
4. **RoPE placement:** prove that values remain unchanged by the rotation helper.
5. **SwiGLU count:** calculate $3CM$ and match the module parameter count.
6. **Full causality:** perturb future tokens and verify earlier block outputs are fixed.

10.8 Hands-on work

Use `exercises/day10_llama_parts.py`. The core Light Demo ends after component-level checks.

1. Implement RMSNorm without calling `nn.LayerNorm`.
2. Build RoPE sine/cosine tables and test norm preservation.
3. Replace the GELU MLP with SwiGLU and select a parameter-matched hidden width.

4. Produce a table comparing shapes and parameter counts before and after the substitutions.
5. **Optional Heavy:** insert RoPE into Day 7 attention and retain all causality tests.

10.9 Weekly acceptance

1. On a blank page, label every shape from token IDs to multi-head attention output.
2. Derive why the RoPE query-key dot product depends on relative displacement.
3. Explain precisely how RMSNorm differs from LayerNorm.
4. Explain why a gated feed-forward layer uses three matrices.
5. Retain the Day 9 Tiny GPT overfit result as the Week 2 end-to-end evidence.
6. Compare a simple GPT block with a LLaMA-style block without claiming all releases use one fixed configuration.

Part III

LoRA, QLoRA, and QA Adaptation

Chapter 11

LoRA: Low-Rank Adaptation

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- state the low-rank assumption made by LoRA;
- derive adapter shapes and trainable-parameter counts;
- implement a LoRA-wrapped linear layer from first principles;
- test initialization, freezing, gradients, and weight merging;
- explain rank, scaling, dropout, and target-module choices.

11.1 Five-hour plan

Time	Activity
60 min	Motivation, low-rank updates, target modules, and inference behavior.
75 min	Shapes, parameter counts, initialization, and gradient derivation.
110 min	Implement and test LoRA on linear layers and Tiny GPT.
35 min	Read the LoRA abstract, method, and main ablations.
20 min	Acceptance questions and experiment log.

11.2 The adaptation problem

Full fine-tuning updates every parameter of a pretrained model. This can be expensive to store, optimize, and distribute when many downstream tasks are required. LoRA freezes a pretrained matrix W_0 and represents the task-specific update with two smaller matrices [18].

For a linear map $y = W_0 x + b$ with

$$W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}},$$

LoRA defines

$$W = W_0 + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA, \tag{11.1}$$

where

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

The forward pass becomes

$$y = W_0 x + b + \frac{\alpha}{r} B(Ax). \tag{11.2}$$

Key idea - The update is low rank

LoRA does not approximate W_0 by a low-rank matrix. It constrains the learned change ΔW to rank at most r while preserving the complete frozen base matrix.

11.3 Parameter count

Full adaptation of W_0 requires $d_{\text{out}} d_{\text{in}}$ trainable weights. LoRA requires

$$N_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}}). \tag{11.3}$$

The trainable fraction for this matrix is

$$\rho = \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}.$$

For a square 4096×4096 projection with $r = 8$, full adaptation uses 16,777,216 weights, while LoRA uses 65,536, about 0.39% as many.

This saving concerns trainable parameters and optimizer states. The frozen base weights are still needed for the forward pass and must be stored somewhere.

11.4 Initialization and initial equivalence

One factor is commonly initialized randomly and the other to zero. If $B = 0$, then $BA = 0$ and the adapted model initially matches the base model exactly. Both factors must not be zero, because then the first-step gradients of both factors vanish.

Let $g = \partial \mathcal{L} / \partial (BA)$. Matrix calculus gives, up to the scale α/r,

$$\frac{\partial \mathcal{L}}{\partial B} = g A^\top, \qquad \frac{\partial \mathcal{L}}{\partial A} = B^\top g.$$

At $B = 0$ and random A, B receives a gradient on the first step while A initially receives zero; after B moves, both can learn.

11.5 A teaching implementation

```python
class LoRALinear(nn.Module):
    def __init__(self, base, rank=8, alpha=16.0, dropout=0.0):
        super().__init__()
        if not isinstance(base, nn.Linear):
            raise TypeError("base must be nn.Linear")
```

```
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False

        self.rank = rank
        self.scale = alpha / rank
        self.dropout = nn.Dropout(dropout)
        self.A = nn.Parameter(torch.empty(rank, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, rank))
        nn.init.kaiming_uniform_(self.A, a=5 ** 0.5)

    def forward(self, x):
        update = (self.dropout(x) @ self.A.T) @ self.B.T
        return self.base(x) + self.scale * update
```

The two-stage multiplication avoids explicitly forming BA during training. Check orientation carefully: for x ending in d_{in}, $xA^\top$ ends in r, and the following product with $B^\top$ ends in d_{out}.

11.6 Rank, alpha, and dropout

The rank r controls the maximum rank and parameter count of the update. A larger rank increases capacity but does not guarantee better validation performance. The hyperparameter α controls update scale through α/r in the original formulation. When comparing ranks, state whether α is fixed, proportional to r, or tuned independently.

LoRA dropout is applied on the adapter branch, not on the frozen base result. It is active only during training. A fair ablation keeps this choice and the optimization budget consistent.

11.7 Choosing target modules

In a Transformer, common targets include query, key, value, attention-output, and feed-forward projections. Target names differ between implementations: one model may expose a combined `qkv` layer while another exposes `q_proj`, `k_proj`, and `v_proj`. Never copy target-module strings without inspecting the model.

Target selection changes both capacity and storage. Record:

- exact module names and count;
- each module's input and output dimensions;
- rank, alpha, and dropout;
- total and trainable parameter counts;
- whether biases or normalization parameters are trained.

11.8 Merging for inference

Because both branches are linear, the adapter can be merged:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA. \tag{11.4}$$

The merged layer has no additional linear-layer latency. Merging should be performed in a suitable precision and only once; repeated merging adds the update repeatedly. Keep metadata that records merge state.

```
@torch.no_grad()
def merged_weight(layer):
    delta = layer.scale * (layer.B @ layer.A)
    return layer.base.weight + delta.to(layer.base.weight.dtype)
```

11.9 Required tests

1. **Initial equivalence:** before training, wrapped and base outputs match.
2. **Frozen base:** base parameters have `requires_grad=False` and no gradients.
3. **Adapter gradients:** after suitable steps, both A and B receive finite gradients.
4. **Count:** trainable parameters equal $r(d_{\text{in}} + d_{\text{out}})$ per wrapped matrix.
5. **Merge equivalence:** merged and unmerged evaluation outputs match within tolerance.
6. **Serialization:** reloaded adapter weights reproduce the same output with the same base revision.

11.10 Hands-on work

Use `exercises/day11_lora.py` and `src/dl100h/lora.py`.

1. Implement the wrapper without a PEFT library.
2. Verify all six tests above on rectangular and square matrices.
3. Add LoRA to Tiny GPT's combined QKV or output projection.
4. Print every trainable parameter name and confirm the base is frozen.
5. Read the LoRA paper and record its problem, assumption, method, evidence, and limitations.

11.11 Acceptance questions

1. Why is LoRA not a low-rank approximation of W_0 itself?
2. What are the shapes of A, B, and BA?
3. Why should only one factor be zero-initialized?
4. How do rank and target-module choice affect trainable parameters?
5. What must be saved besides adapter tensors to reproduce inference?

Chapter 12

QLoRA, Quantization Error, and Memory Accounting

Study level: Light Demo

Run fake-quantization and memory-ledger exercises. Actual 4-bit large-model fine-tuning is Optional Heavy.

Learning objectives

By the end of this chapter, you should be able to:

- state exactly what is quantized, frozen, and trained in QLoRA;
- distinguish storage precision from compute precision;
- implement symmetric fake quantization and measure its error;
- explain NF4, double quantization, and paged optimizers;
- build a defensible training-memory ledger.

12.1 Five-hour plan

Time	Activity
70 min	Quantization concepts and the QLoRA training path.
65 min	Error derivation and theoretical memory accounting.
90 min	Lightweight fake-quantization and memory-estimator exercises.
45 min	Read the QLoRA abstract, method figure, and ablations.
30 min	Acceptance questions and memory ledger.

12.2 What QLoRA actually trains

QLoRA stores a pretrained base model in a frozen 4-bit quantized representation and backpropagates through dequantized computations into trainable LoRA adapters [19]. The base weights are not updated as ordinary 4-bit trainable parameters.

Object	Stored as	Receives gradients?	Updated?
Base weights	quantized 4-bit plus metadata	no	no
LoRA adapters	usually bf16/fp16/fp32	yes	yes
Activations	compute-dependent precision	through graph	not parameters
Optimizer state	adapter-dependent	not applicable	yes

During a matrix multiplication, blocks of quantized base weights are dequantized to a higher compute type. Therefore, "4-bit model" describes weight storage, not that every arithmetic operation occurs in 4-bit.

12.3 Uniform symmetric fake quantization

To understand quantization, begin with a simple signed uniform scheme. For b bits, let

$$q_{\max} = 2^{b-1} - 1, \qquad s = \frac{\max_i |w_i|}{q_{\max}}.$$

Quantize and dequantize by

$$q_i = \operatorname{clip}\left(\operatorname{round}(w_i/s), -q_{\max}, q_{\max}\right), \tag{12.1}$$

$$\widehat{w}_i = s q_i. \tag{12.2}$$

If rounding occurs without clipping, the absolute error is bounded by about $s/2$.

```
def symmetric_fake_quantize(w, bits):
    qmax = 2 ** (bits - 1) - 1
    scale = w.abs().max() / max(qmax, 1)
    if scale == 0:
        return w.clone()
    q = torch.round(w / scale).clamp(-qmax, qmax)
    return q * scale
```

This is a teaching approximation. It is not NF4 and does not reproduce the kernels, block structure, metadata, or numerical behavior of a QLoRA implementation.

Measure at least mean squared error,

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(w_i - \widehat{w}_i)^2,$$

maximum absolute error, and task-output error. Small weight MSE does not automatically imply unchanged model quality.

12.4 NF4

NormalFloat 4-bit, or NF4, uses a nonuniform codebook designed for approximately normally distributed pretrained weights. More representational levels are allocated where probability mass is high. Weights are quantized in blocks, each with a scale or related quantization constant.

The advantage depends on distributional assumptions and implementation details. Do not describe NF4 as merely "ordinary int4." Its codebook and blockwise normalization are essential parts of the method.

12.5 Double quantization and paged optimizers

Blockwise quantization requires metadata such as scales. Double quantization compresses these quantization constants, reducing average storage overhead. It does not mean that the same model weights are arbitrarily quantized twice.

Paged optimizers use unified-memory mechanisms to handle temporary memory spikes more gracefully. They address peak-memory behavior; they do not make activation memory disappear or remove the need to size a workload conservatively.

12.6 Weight-storage floor

For N parameters stored with b bits each, the idealized weight-only floor is

$$M_{\text{weights}} = \frac{Nb}{8} \text{ bytes.} \tag{12.3}$$

For $N = 7 \times 10^9$:

Format	Decimal GB	Binary GiB
fp32	28.0	26.08
fp16/bf16	14.0	13.04
8-bit	7.0	6.52
4-bit ideal floor	3.5	3.26

Real storage exceeds the ideal 4-bit floor because of quantization metadata and nonquantized tensors.

12.7 The training-memory ledger

Peak training memory includes more than weights:

$$M_{\text{peak}} \approx M_{\text{base}} + M_{\text{adapter}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{activation}} + M_{\text{temporary}}. \tag{12.4}$$

For P trainable adapter parameters, an Adam-style estimate might include adapter weights, gradients, and two optimizer moments. The exact bytes depend on whether master weights and moments use fp32 or lower precision. Activation memory depends on batch size, sequence length, width, depth, attention implementation, checkpointing, and saved intermediates.

Common pitfall

Do not advertise the weight-only 4-bit floor as the GPU requirement for QLoRA training. Report both a theoretical floor and a measured peak, together with the full configuration.

12.8 How to measure peak memory

On CUDA, reset and read peak allocated memory around the complete training step:

```
torch.cuda.empty_cache()
torch.cuda.reset_peak_memory_stats()

loss = training_step(batch)
loss.backward()
optimizer.step()

peak = torch.cuda.max_memory_allocated() / 2**30
print(f"peak allocated: {peak:.2f} GiB")
```

Also record reserved memory if relevant, GPU model, software versions, dtype, batch size, accumulation steps, sequence length, and whether activation checkpointing is enabled.

12.9 Hands-on work

Use `exercises/day12_quantization.py` and `scripts/vram_estimator.py`.

1. Compare fake 2-, 4-, and 8-bit quantization on normal and heavy-tailed matrices.
2. Report weight error and downstream linear-layer output error.
3. Compute decimal GB and binary GiB for 1B, 7B, and 13B parameter bases.
4. Build a theoretical-floor versus measured-peak table.
5. Map NF4, double quantization, and paged optimizers to the problem each addresses.

12.10 Acceptance questions

1. Are the pretrained base weights updated in QLoRA?
2. Which parameters receive optimizer states?
3. Why is storage precision different from compute precision?
4. What overhead is missing from the ideal $Nb/8$ calculation?
5. Why is symmetric fake quantization not an implementation of NF4?

Chapter 13

Optional Hugging Face and PEFT Engineering Path

Study level: Understand Only

Trace the configuration, module mapping, data flow, and saving contract. Installing heavy-weight dependencies or training a remote model is not required.

Learning objectives

By the end of this chapter, you should be able to:

- map the teaching implementation to Transformers, Datasets, PEFT, and Accelerate;
- specify a reproducible LoRA or QLoRA configuration;
- inspect target modules instead of guessing their names;
- save the adapter, tokenizer, configuration, and base revision together;
- choose a safe lightweight or optional GPU execution route.

13.1 Five-hour plan

Time	Activity
60 min	Library roles, configuration, licensing, and reproducibility.
60 min	Trainable-parameter and memory calculations for a chosen model.
120 min	Optional PEFT run, or an offline configuration and code review.
30 min	Audit the supplied GPT-2/LLaMA QA LoRA notebook.
30 min	Acceptance questions and run manifest.

13.2 From equations to libraries

The lightweight implementation from Day 11 remains the conceptual reference. Engineering libraries add model registries, configuration, serialization, device handling, data pipelines, and optimized kernels.

Layer	Responsibility
Transformers	tokenizer, architecture class, pretrained configuration and weights
Datasets	versioned loading, transforms, caching, and split operations
PEFT	adapter injection, trainable-parameter control, saving, loading, and merging
Accelerate	device placement and distributed or mixed-precision execution
bitsandbytes	supported low-bit linear layers and optimizer implementations

PEFT uses a configuration object to describe rank, alpha, dropout, task type, and target modules, then applies that configuration to a base model. Its documentation emphasizes that the adapter configuration is required when reloading an adapter [20].

13.3 Preflight before downloading a model

Record these items before starting:

1. model ID and immutable revision or commit hash;
2. license and whether access approval is required;
3. tokenizer ID and revision;
4. parameter count, expected dtype, and estimated memory;
5. task type and expected output format;
6. dataset source, version, license, and split policy;
7. whether remote custom code is required and why it is trusted.

Do not place access tokens in notebooks, source files, screenshots, or experiment logs. Use the authentication mechanism provided by the environment.

13.4 Inspect before injecting adapters

Module names are architecture-specific. Print candidate modules and dimensions:

```
for name, module in model.named_modules():
    if isinstance(module, torch.nn.Linear):
        print(name, module.in_features, module.out_features)
```

Then select targets deliberately. GPT-2-style `Conv1D` projections can store weight orientation differently from `nn.Linear`; PEFT exposes configuration options for such cases. Confirm the model-specific convention rather than assuming one layout [21].

13.5 A documented LoRA configuration

The following is a pattern, not a universal prescription:

```
from peft import LoraConfig, TaskType, get_peft_model

config = LoraConfig(
    r=8,
    lora_alpha=16,
```

```
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],  # verify for this model
    bias="none",
    task_type=TaskType.CAUSAL_LM,
)
model = get_peft_model(base_model, config)
model.print_trainable_parameters()
```

After injection, audit the result:

- list all trainable parameter names;
- compare reported and manually predicted counts;
- verify that intended targets are wrapped exactly once;
- run initial-equivalence and gradient tests when supported;
- save the resolved configuration with the run.

Common pitfall

Adapter injection can mutate a model object. For a new configuration or rank ablation, reload a fresh base model rather than repeatedly wrapping an already modified instance.

13.6 A documented 4-bit configuration

The Transformers integration exposes configuration for 4-bit loading, compute dtype, NF4, and nested quantization [23]. A typical configuration shape is:

```
from transformers import BitsAndBytesConfig

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)
```

Hardware and software support varies. Treat any example memory figure as configuration-specific. Verify the current official compatibility documentation for the actual machine before installing the heavyweight dependencies.

13.7 Data and collation

A causal language-model training example commonly contains prompt and answer tokens in one sequence. Define which positions contribute to loss. If prompt tokens are masked, their labels use the ignored target ID; answer positions retain their token IDs.

```
labels = input_ids.clone()
labels[:prompt_length] = -100
```

Document truncation side, maximum length, padding side, special-token insertion, and whether examples that lose the answer during truncation are rejected. Decode several tokenized examples before training.

13.8 Saving a reproducible adapter

An adapter checkpoint is not self-sufficient unless the base and preprocessing contract are known. Save:

- adapter weights and adapter configuration;
- base-model ID and immutable revision;
- tokenizer files and special-token changes;
- model and quantization configuration;
- prompt template and label-masking policy;
- package versions, seed, and training arguments;
- evaluation normalization rules.

PEFT provides `save_pretrained` and corresponding loading APIs, but the experiment manifest must still capture the external dependencies [22].

13.9 Two valid execution routes

Lightweight route

Do not download a large model. Instead, inspect a saved configuration, calculate parameter counts, validate data formatting with a tiny local model, and audit the supplied notebook. This route is sufficient for the conceptual acceptance criteria.

Optional GPU route

Install the separately listed heavyweight dependencies, choose a small public model with a clear license, and first overfit eight examples. Record measured peak memory. Stop if the environment cannot safely support the selected configuration.

13.10 Lightweight verification

Run `exercises/day13_hf_peft_optional.py` to inventory optional packages without downloading a model. The Understand Only route requires a completed configuration manifest and data-flow trace, not installation of every listed package.

13.11 Notebook audit

For the supplied `LLM_GPT2_LLaMA_QA_LoRA_FINAL.ipynb` [24], record without copying large code sections:

1. model and tokenizer identifiers;
2. task formulation and prompt format;
3. adapter target modules, rank, alpha, and dropout;
4. which parameters are frozen and trainable;

5. dataset source and split method;
6. metric definitions and normalization;
7. hardware assumptions and peak-memory evidence;
8. one improvement needed for reproducibility.

13.12 Acceptance questions

1. Why must target-module names be inspected for each architecture?
2. Which configuration fields are required to interpret a LoRA checkpoint?
3. Why must tokenizer and adapter configuration be saved?
4. What does a lightweight configuration review prove, and what does it not prove?
5. Which facts must accompany a claimed QLoRA memory requirement?

Chapter 14

Question Answering Data, Metrics, and Split Auditing

Study level: Core Run

Complete the lightweight local exercise and pass the behavioral checks. No external model or dataset is required.

Learning objectives

By the end of this chapter, you should be able to:

- distinguish extractive, generative, and closed-answer QA;
- implement transparent Exact Match and token F1 metrics;
- define normalization without hiding clinically meaningful errors;
- report stratified metrics and simple baselines;
- audit duplicate, template, image, and patient leakage across splits.

14.1 Five-hour plan

Time	Activity
60 min	QA formulations, answer spaces, normalization, and baselines.
55 min	Derive EM, precision, recall, token F1, and majority accuracy.
120 min	Implement metrics, confusion matrices, and split audits.
35 min	Read dataset documentation and record evaluation assumptions.
30 min	Acceptance questions and failure-case table.

14.2 Three QA formulations

Extractive QA predicts a span from a supplied text context. The output space consists of start and end positions. **Closed-answer classification** predicts one label from a fixed answer vocabulary. **Generative QA** emits a variable-length token sequence.

Medical visual QA is often evaluated as classification, generation, or both. The formulation affects model architecture, handling of rare answers, decoding, and metrics. Do not compare results from different output spaces as though only the model changed.

14.3 Answer normalization

Exact string comparison is sensitive to case, punctuation, articles, whitespace, units, spelling variants, and synonyms. A normalization function might lowercase, standardize whitespace, or remove selected punctuation. Every rule changes the metric and must be documented and tested.

In medical QA, aggressive normalization can be dangerous. "Positive" and "negative," left and right, units, or severity qualifiers are not interchangeable. Synonym mapping should come from a declared vocabulary or expert-reviewed rule set rather than convenient post-hoc edits.

```
import re
import string

def normalize_answer(text):
    text = text.lower().strip()
    text = " ".join(text.split())
    text = re.sub(f"[{re.escape(string.punctuation)}]", " ", text)
    text = re.sub(r"\b(a|an|the)\b", " ", text)
    return " ".join(text.split())
```

This baseline removes common English articles but intentionally does not remove numbers, units, negation, or medical terms. Article removal and every other rule must be declared in the evaluation protocol.

14.4 Exact Match

For normalized prediction $\widehat{a}_i$ and one reference a_i,

$$\mathrm{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\widehat{a}_i = a_i]. \tag{14.1}$$

With multiple acceptable references, score against each and take the maximum per example. EM is strict and interpretable, but gives zero credit to a partially correct multi-token answer.

14.5 Token F1

Treat normalized tokens as multisets so repeated tokens are counted correctly. Let m be the multiset intersection size, n_p the predicted-token count, and n_r the reference-token count:

$$P = \frac{m}{n_p}, \qquad R = \frac{m}{n_r}, \tag{14.2}$$

$$F_1 = \frac{2PR}{P+R}. \tag{14.3}$$

Define edge cases explicitly: two empty answers may score one, while exactly one empty answer scores zero.

Example: prediction “mild left pleural effusion” and reference “left pleural effusion” have EM = 0, precision = 3/4, recall = 1, and F1 = 6/7. Token overlap reflects partial agreement but does not prove clinical equivalence.

```python
from collections import Counter

def token_f1(prediction, reference):
    pred = normalize_answer(prediction).split()
    ref = normalize_answer(reference).split()
    if not pred or not ref:
        return float(pred == ref)
    overlap = sum((Counter(pred) & Counter(ref)).values())
    if overlap == 0:
        return 0.0
    precision = overlap / len(pred)
    recall = overlap / len(ref)
    return 2 * precision * recall / (precision + recall)
```

14.6 Baselines and stratification

For a closed yes/no dataset with positive proportion p, always predicting the majority class achieves

$$\text{Accuracy}_{\text{majority}} = \max(p, 1 - p). \tag{14.4}$$

Report this baseline and class counts. For answer-vocabulary classification, also report the most-frequent-answer baseline.

Aggregate scores can hide failures. Stratify at least by question type, answer type, answer frequency, and relevant image or clinical category. Always include sample counts and, where practical, uncertainty intervals or variation across seeds.

14.7 Split leakage

Random row splitting is unsafe when examples share patients, images, reports, templates, or near-duplicate questions. Leakage can make validation look like retrieval of training patterns.

Audit intersections of:

- exact normalized question text;
- question-answer pairs;
- image identifiers and hashes;
- patient, study, or case identifiers;
- normalized templates after replacing entities;
- near-duplicate text detected with a documented threshold.

If multiple observations belong to one patient or study, split by that group identifier before model development. Keep the test split isolated from prompt, normalization, and hyperparameter decisions.

```python
def overlap_report(train, valid, key):
    train_ids = {row[key] for row in train}
    valid_ids = {row[key] for row in valid}
    shared = train_ids & valid_ids
    return {"key": key, "count": len(shared), "examples": list(shared)[:5]}
```

14.8 Failure-case records

For every reviewed error, store:

- example, image, patient/study, and split identifiers;
- question, normalized question, target references, and prediction;
- question and answer types;
- raw score or confidence when meaningful;
- EM and F1;
- attribution: data, normalization, visual evidence, reasoning, decoding, or annotation ambiguity;
- reviewer note without protected or identifying information.

14.9 Hands-on work

Use `exercises/day14_qa_metrics.py`.

1. Implement normalization, EM, and multiset token F1 with unit tests.
2. Create an example for every normalization rule and edge case.
3. Produce a confusion matrix for a closed-answer subset.
4. Audit question, image, and patient/study overlap between splits.
5. Build a failure table and summarize the three most common error types.

14.10 Acceptance questions

1. Why is EM useful but insufficient?
2. How can over-aggressive normalization inflate performance?
3. Why must token overlap use multiset rather than set intersection?
4. How do classification and generation output spaces differ?
5. Which grouping key should control a medical-data split, and why?

Chapter 15

LoRA Rank-Ablation Mini-Project

Study level: Light Demo

Complete a small synthetic rank comparison. Full multi-seed language-model ablation is Optional Heavy.

Learning objectives

By the end of this chapter, you should be able to:

- formulate a falsifiable hypothesis about LoRA rank;
- design a controlled rank ablation with a fixed training budget;
- separate approximation capacity from optimization and seed variation;
- record parameters, metrics, runtime, and peak memory consistently;
- select a rank using evidence rather than the largest score alone.

15.1 Five-hour plan

Time	Activity
35 min	Define question, controls, hypotheses, and success criteria.
35 min	Derive rank capacity and parameter growth.
170 min	Run the lightweight ablation and analyze results.
25 min	Review related LoRA or QA evidence.
35 min	Weekly acceptance and final experiment record.

15.2 Research question

For the Light Demo, compare two ranks, for example

$$r \in \{2, 8\}.$$

The Optional Heavy extension uses $r \in \{2, 4, 8, 16\}$ and repeated seeds. Keep the base model, target modules, data split, preprocessing, optimizer family, step budget, and evaluation protocol fixed.

The lightweight default is a synthetic low-rank linear adaptation task. With suitable hardware, the same design can be applied to a small text QA model, but the lightweight experiment remains the reproducible baseline.

The question is not "Which run has the largest metric?" It is:

> How does adapter rank change approximation capacity, trainable parameters, optimization behavior, validation performance, runtime, and memory under a controlled budget?

15.3 Rank and approximation error

Suppose the true task change is ΔW^* with rank r^*. A LoRA update BA has rank at most r. If $r < r^*$, the update cannot represent ΔW^* exactly. The best rank-r approximation under Frobenius norm is given by the truncated singular value decomposition:

$$\min_{\operatorname{rank}(M)\le r} \|\Delta W^* - M\|_F^2 = \sum_{i>r} \sigma_i^2, \tag{15.1}$$

where σ_i are singular values of ΔW^*.

When $r \ge r^*$, representational capacity is sufficient in principle. Training may still fail because of optimization, regularization, finite data, or an unfair budget. Larger rank can fit training data better while leaving validation unchanged or worse.

For one adapted matrix, trainable parameters grow linearly:

$$N(r) = r(d_{\text{in}} + d_{\text{out}}).$$

Across targets j,

$$N_{\text{total}}(r) = r \sum_j (d_{\text{in},j} + d_{\text{out},j}).$$

15.4 Write hypotheses first

Before running code, write one or two sentences for each prompt:

1. Predict how training error will change as rank increases, and explain why.
2. Predict whether validation performance will plateau after some rank.
3. State the expected relationship between rank and trainable parameters.
4. Explain why measured peak memory may not grow perfectly linearly.
5. Define the rule that will select the smallest acceptable rank.

These predictions prevent the explanation from being rewritten to fit the observed curves.

15.5 Experimental controls

Keep fixed:

- base initialization or exact pretrained revision;
- train, validation, and test splits;

- tokenizer, maximum length, and label masking;
- target modules, alpha policy, dropout, and bias policy;
- optimizer, scheduler, batch size, accumulation, and maximum steps;
- checkpoint-selection metric and evaluation frequency;
- generation and normalization settings.

Rank is the intended independent variable. If alpha is set proportional to rank, say so; if alpha is fixed, say so. These policies change effective scaling.

15.6 Seeds and fair stopping

For the Light Demo, one seed is acceptable if the report explicitly says that seed variation was not estimated. The Optional Heavy route uses at least three seeds per rank when feasible and reports mean and standard deviation:

$$\bar{m} = \frac{1}{S}\sum_{s=1}^{S} m_s, \qquad s_m = \sqrt{\frac{1}{S-1}\sum_{s=1}^{S}(m_s - \bar{m})^2}.$$

If the difference between two ranks is smaller than seed variation, do not claim a reliable improvement.

Use the same maximum update count. If early stopping is enabled, use identical patience and evaluation frequency, and report both the selected step and total compute. A run that receives more opportunities is not a fair comparison.

15.7 Run manifest

Record one row per rank and seed:

Rank	Seed	Trainable	Best step	Val metric	Time	Peak GiB	Note
2							
8							
4 (optional)							
16 (optional)							

Also keep final training metric, checkpoint path, adapter size, base revision, and code revision. Retain failed and unattractive runs.

15.8 Lightweight synthetic task

Construct a frozen base W_0 and true task update $\Delta W^* = B^*A^*$ of known rank. Generate

$$y = (W_0 + \Delta W^*)x + \epsilon.$$

Fit only a LoRA update at several ranks. This experiment isolates low-rank approximation from tokenization and language-model complexity.

```python
def make_task(d_in, d_out, true_rank, n, noise, generator):
    W0 = torch.randn(d_out, d_in, generator=generator)
    A = torch.randn(true_rank, d_in, generator=generator)
    B = torch.randn(d_out, true_rank, generator=generator)
    X = torch.randn(n, d_in, generator=generator)
    Y = X @ (W0 + B @ A).T
    Y = Y + noise * torch.randn(Y.shape, generator=generator)
    return W0, X, Y
```

Use separate generated train and validation samples from the same known process. Compare the learned ΔW with ΔW^* by validation MSE and matrix error.

15.9 Hands-on work

Use `exercises/day15_lora_adaptation.py` as the single-rank sanity check. Extend it to compare $r = 2$ and $r = 8$ on separate generated training and validation samples, then record the Light Demo result table. Repeated seeds, four ranks, plots, and language-model adaptation belong to the Optional Heavy extension.

15.10 Results to record

For the Light Demo, record a table containing rank, trainable parameters, final training loss, validation loss, and runtime. The Optional Heavy extension adds:

1. rank versus trainable-parameter count;
2. rank versus validation metric with seed variation;
3. optimization curves using a common horizontal axis;
4. rank versus runtime and measured peak memory.

Do not use a smoothed curve that hides individual seeds. Include units and state whether memory is allocated, reserved, or device-wide.

15.11 Selecting a rank

The best rank is often a Pareto choice rather than the maximum rank. Prefer a smaller rank when it reaches the performance tolerance with materially fewer parameters, lower adapter storage, faster training, or more stable results. A larger rank is justified when its gain exceeds seed variation and matters for the intended task.

Checkpoint

Define the selection rule before reading the test split. For example: choose the smallest rank whose mean validation score is within one standard deviation of the best mean, then evaluate that selected configuration once on the test set.

15.12 Failure analysis

Interpret negative results carefully:

- all ranks fail: inspect data, gradients, target modules, learning rate, and base suitability;
- train improves but validation does not: consider overfitting or split shift;
- high rank underperforms: check scaling, optimization, and regularization;
- memory barely changes: frozen-base or activation memory may dominate;
- large seed variation: extend runs or weaken the conclusion.

15.13 Weekly acceptance

1. State what is frozen, quantized, dequantized, and trained in QLoRA.
2. Demonstrate LoRA initial equivalence, frozen gradients, and merge equivalence.
3. Complete a lightweight synthetic comparison with at least two ranks.
4. Treat three or more seeds and a language-model ablation as Optional Heavy; otherwise state that seed variation was not estimated.
5. Explain the mapping from LoRA equations to QLoRA storage and PEFT abstractions.
6. Preserve and discuss at least one negative or inconclusive result.

Part IV

ViT, CLIP, VLM, and Medical PathVQA

Chapter 16

Vision Transformer

Study level: Light Demo

Trace patch shapes and run a tiny local ViT demonstration. Pretrained large-scale vision training is not required.

Learning objectives

- convert images into patch-token sequences and trace every shape;
- explain the equivalence between patch projection and strided convolution;
- calculate token count, projection parameters, and attention cost;
- compare class-token and mean pooling;
- explain why patch size and resolution matter for small targets.

16.1 Five-hour plan

65 min concepts; 60 min derivation; 115 min implementation; 35 min ViT paper; 25 min acceptance.

16.2 Images as token sequences

For an image $X \in \mathbb{R}^{B\times C_{\text{in}}\times H\times W}$ and square patch size P dividing H, W, the patch count is

$$N = \frac{H}{P}\frac{W}{P}. \tag{16.1}$$

Each flattened patch has P^2C_{in} values and is projected to model width D. ViT adds position embeddings and optionally prepends a learned class token before applying a Transformer encoder [25].

$$\begin{aligned} \text{image} &: (B, C_{\text{in}}, H, W), \\ \text{patch vectors} &: (B, N, P^2C_{\text{in}}), \\ \text{patch tokens} &: (B, N, D), \\ \text{with class token} &: (B, N+1, D). \end{aligned}$$

16.3 Convolutional patch projection

A `Conv2d` with kernel size P, stride P, and D output channels computes one learned linear projection per nonoverlapping patch. Its weight shape is

$$(D, C_{\text{in}}, P, P),$$

so the parameter count including bias is

$$D(C_{\text{in}}P^2) + D. \tag{16.2}$$

```python
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels, width, patch):
        super().__init__()
        self.proj = nn.Conv2d(
            in_channels, width, kernel_size=patch, stride=patch
        )

    def forward(self, images):
        x = self.proj(images)       # (B, D, H/P, W/P)
        return x.flatten(2).transpose(1, 2)  # (B, N, D)
```

16.4 Patch size trade-off

Halving P doubles the grid resolution in each spatial dimension and therefore multiplies N by four. Self-attention score memory scales as N^2, so this change can increase score elements by sixteen. Smaller patches preserve finer spatial detail but make global attention much more expensive.

For medical images, an abnormal region smaller than or comparable to a patch can be blurred by resize, crop, compression, or patch projection. Record original resolution, preprocessing resolution, physical scale when available, patch size, and augmentation policy.

Common pitfall

Increasing input resolution does not automatically preserve evidence. Inspect interpolation, crop policy, aspect ratio, and whether the clinically relevant region is still present after preprocessing.

16.5 Position and pooling

Learned absolute position embeddings have shape $(1, N + 1, D)$ for a class-token model. Changing resolution changes N and may require interpolation or a compatible positional scheme. State exactly how pretrained positions are adapted.

Class-token pooling uses the final representation of one learned token. Mean pooling averages patch representations. Neither is universally superior; compare them with identical encoders and training budgets.

16.6 Inductive bias

CNNs encode locality and translation-related structure through small shared kernels. A basic ViT has weaker built-in locality: global attention can connect distant patches immediately, but it often relies more heavily on data, augmentation, or pretraining. This is a difference in inductive bias, not proof that either family is always better.

16.7 Minimal classifier

```python
class TinyViT(nn.Module):
    def __init__(self, image_size, patch, classes, width=128, layers=4):
        super().__init__()
        self.patch = PatchEmbedding(3, width, patch)
        n = (image_size // patch) ** 2
        self.cls = nn.Parameter(torch.zeros(1, 1, width))
        self.pos = nn.Parameter(torch.zeros(1, n + 1, width))
        block = nn.TransformerEncoderLayer(
            width, nhead=4, batch_first=True, norm_first=True
        )
        self.encoder = nn.TransformerEncoder(block, layers)
        self.head = nn.Linear(width, classes)

    def forward(self, images):
        x = self.patch(images)
        cls = self.cls.expand(images.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos
        return self.head(self.encoder(x)[:, 0])
```

This is a teaching model, not an official reproduction of a particular pretrained ViT.

16.8 Hands-on work

Use `exercises/day16_vit.py` and `src/dl100h/vision.py`.

1. Verify patch order with an image whose patches contain known constants.
2. Change patch size from 8 to 4 and record token count, score size, and runtime.
3. Compare class-token and mean pooling under identical conditions.
4. Overfit one tiny image batch and confirm gradients reach patch projection.
5. Write a preprocessing audit for one pathology-style image.

16.9 Acceptance questions

1. Derive token count and attention-score growth when patch size is halved.
2. How do ViT and CNN inductive biases differ?
3. Why are small targets sensitive to resize and patch size?
4. Why can a strided convolution implement patch projection?

5. What must change when inference resolution differs from pretraining resolution?

Chapter 17

CLIP and Image-Text Contrastive Learning

Study level: Light Demo

Derive the contrastive loss and overfit a tiny synthetic batch. Large-scale CLIP training is not required.

Learning objectives

- derive the symmetric image-text contrastive objective;
- interpret the batch similarity matrix and temperature;
- implement retrieval metrics and alignment tests;
- explain false negatives and batch-composition effects;
- describe prompt-based zero-shot classification.

17.1 Five-hour plan

60 min concepts; 70 min derivation; 110 min coding; 35 min CLIP paper; 25 min acceptance.

17.2 Two encoders, one space

CLIP trains an image encoder f_I and a text encoder f_T on paired examples, then normalizes the embeddings [26]:

$$u_i = \frac{f_I(I_i)}{\|f_I(I_i)\|_2}, \qquad v_i = \frac{f_T(T_i)}{\|f_T(T_i)\|_2}.$$

For batch size B, the similarity logits are

$$S_{ij} = \frac{u_i^\top v_j}{\tau}, \tag{17.1}$$

forming a (B, B) matrix. The diagonal contains paired examples; off-diagonal entries are treated as negatives.

17.3 Symmetric loss

Image-to-text cross-entropy treats each row as a classification problem, while text-to-image treats each column similarly:

$$\mathcal{L}_{I\to T} = -\frac{1}{B}\sum_i \log \frac{\exp S_{ii}}{\sum_j \exp S_{ij}}, \tag{17.2}$$

$$\mathcal{L}_{T\to I} = -\frac{1}{B}\sum_i \log \frac{\exp S_{ii}}{\sum_j \exp S_{ji}}, \tag{17.3}$$

$$\mathcal{L} = \tfrac{1}{2}(\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I}). \tag{17.4}$$

```
def clip_loss(image_features, text_features, logit_scale):
    image_features = nn.functional.normalize(image_features, dim=-1)
    text_features = nn.functional.normalize(text_features, dim=-1)
    logits = logit_scale.exp() * image_features @ text_features.T
    labels = torch.arange(logits.size(0), device=logits.device)
    i2t = nn.functional.cross_entropy(logits, labels)
    t2i = nn.functional.cross_entropy(logits.T, labels)
    return 0.5 * (i2t + t2i), logits
```

Temperature controls distribution sharpness. Implementations often learn a log scale and constrain it to a safe range. Extremely sharp logits can destabilize optimization or encourage brittle matching.

17.4 Retrieval metrics

Loss alone does not show whether correct pairs are ranked first. Image-to-text Recall@k is

$$R@k = \frac{1}{B}\sum_i \mathbf{1}[i \text{ appears among the top-}k \text{ texts for image } i].$$

Report both directions. For duplicated captions or multiple valid matches, define a set of valid positives rather than forcing exactly one diagonal target.

17.5 False negatives and shortcuts

Two different images may share an equally valid caption, or two captions may describe the same concept. Treating these as negatives creates contradictory supervision. Detect exact and semantic duplication, inspect batch construction, and consider multi-positive objectives when the data supports them.

Contrastive models can also exploit acquisition artifacts, text templates, or source-specific patterns. Evaluate under source, patient, or site shifts when available.

17.6 Zero-shot classification

Convert each class c into one or more text prompts such as “an image of c.” Encode and average normalized prompt embeddings. Predict the class with highest image-text similarity. Prompt choice is part of the evaluation protocol; report templates and do not tune them on the test set.

CLIP learns a matching representation. It is not by itself a model that generates answers or explanations.

17.7 Hands-on work

Use `exercises/day17_clip.py`.

1. Make TinyCLIP overfit eight synthetic image-text pairs.
2. Shuffle text order and confirm loss increases and retrieval falls.
3. Duplicate captions and explain the resulting false-negative conflict.
4. Report image-to-text and text-to-image Recall@1 and Recall@5.
5. Compare at least three prompt templates for zero-shot classification.

17.8 Acceptance questions

1. Why does CLIP use both row-wise and column-wise cross-entropy?
2. What role does temperature play?
3. How can duplicate semantics create false negatives?
4. How does zero-shot classification turn classes into text?
5. Why does good retrieval not imply safe medical interpretation?

Chapter 18

Vision-Language Fusion and Generation

Study level: Light Demo

Run the pooled fusion and modality-shuffle demonstrations. Connecting a large language model is Optional Heavy.

Learning objectives

- compare pooled fusion, projected visual tokens, and cross-attention;
- trace cross-attention shapes and complexity;
- distinguish closed-answer classification from generation;
- design modality-shuffle tests for shortcut detection;
- identify which components can be frozen or adapted with LoRA.

18.1 Five-hour plan

65 min concepts; 50 min derivation; 125 min coding; 35 min reading; 25 min acceptance.

18.2 Three fusion patterns

Let image features be $V \in \mathbb{R}^{B\times N_v\times C_v}$ and question tokens be $Q \in \mathbb{R}^{B\times T\times C_l}$.

Pooled fusion combines one image vector and one question vector using concatenation, product, or both, then predicts a closed answer. It is cheap and useful as a baseline.

Projected visual tokens use a map $P : \mathbb{R}^{C_v} \to \mathbb{R}^{C_l}$ so visual tokens can enter a language-model-width sequence. A linear projector has $C_lC_v + C_l$ parameters.

Cross-attention uses language queries and visual keys/values:

$$\text{CrossAttn}(Q, K_V, V_V) = \text{softmax}\left(\frac{QK_V^\top}{\sqrt{D}}\right)V_V. \tag{18.1}$$

With H heads, scores have shape (B, H, T, N_v) and compute is proportional to BTN_vC_l.

Key idea - Alignment is not evidence of grounding

A projector aligns dimensions, and attention permits information flow, but neither guarantees that predictions use the image. Behavioral ablations are required.

18.3 Closed versus generative answers

Closed-answer classification maps fused features to a fixed vocabulary. It is easy to constrain and evaluate but cannot emit unseen answers. Generative VQA predicts answer tokens autoregressively. It supports an open vocabulary but introduces decoding sensitivity, hallucination risk, and normalization ambiguity.

For long-tailed answers, a classifier needs an explicit unknown or out-of-vocabulary policy. Report vocabulary coverage on each split and never silently discard rare validation answers.

18.4 A lightweight fusion baseline

```
class PooledVQA(nn.Module):
    def __init__(self, image_encoder, question_encoder, width, answers):
        super().__init__()
        self.image_encoder = image_encoder
        self.question_encoder = question_encoder
        self.image_proj = nn.Linear(image_encoder.out_dim, width)
        self.question_proj = nn.Linear(question_encoder.out_dim, width)
        self.head = nn.Sequential(
            nn.LayerNorm(2 * width),
            nn.Linear(2 * width, answers),
        )

    def forward(self, images, questions):
        vi = self.image_proj(self.image_encoder(images))
        tq = self.question_proj(self.question_encoder(questions))
        return self.head(torch.cat([vi, tq], dim=-1))
```

This baseline is intentionally interpretable and cheap. Compare it with question-only and image-only versions before adding a large language model.

18.5 Modality-use tests

For a fixed evaluation set:

1. shuffle images while preserving questions;
2. shuffle questions while preserving images;
3. replace images with a constant or noise;
4. mask question content while retaining length and punctuation;
5. compare full, question-only, image-only, and majority baselines.

If image shuffling leaves performance unchanged, the system may be dominated by language priors. A drop is evidence of image dependence, not proof that the correct region was used.

18.6 Freezing and adapters

A practical small-data pipeline may freeze visual and language encoders and train only a projector and answer head. LoRA can adapt selected language or attention projections. Record every trainable module and compare against the same frozen-feature baseline; otherwise increased capacity is confounded with architecture choice.

18.7 Hands-on work

Use `exercises/day18_vlm.py`.

1. Implement question-only, image-only, concatenation, and product baselines.
2. Trace projector and cross-attention shapes.
3. Add image- and question-shuffle ablations.
4. Compare parameter counts and runtime for pooled fusion and cross-attention.
5. Overfit eight multimodal examples before using a full split.

18.8 Acceptance questions

1. Which representation spaces does the visual projector connect?
2. What is the cross-attention score shape?
3. How can language-prior shortcuts be detected?
4. How should a classification vocabulary handle rare answers?
5. Why is attention visualization alone insufficient evidence of grounding?

Chapter 19

PathVQA Data, Metrics, and Medical Boundaries

Study level: Understand + Data Audit

Complete the data card, metric definitions, and leakage audit. Training on real medical data is optional and must respect its license and governance.

Learning objectives

- construct a PathVQA data card and split audit;
- report closed and open-ended questions separately;
- specify majority, question-only, and multimodal baselines;
- categorize failures without making clinical claims;
- write explicit limitations and data-governance statements.

19.1 Five-hour plan

65 min concepts; 45 min derivation; 110 min coding; 40 min paper/notebook audit; 40 min acceptance.

19.2 Dataset context

PathVQA was introduced for visual question answering on pathology images. The published dataset contains thousands of images and tens of thousands of questions, with closed yes/no and open-ended questions [27]. Use the official paper and repository for the exact version and split being analyzed; do not mix counts from mirrors without explanation.

The learning goal here is computational evaluation, not medical diagnosis. Dataset labels derived from captions, textbooks, or curated sources do not establish clinical deployment validity.

19.3 Data card

Document:

- source, version, license, access route, and citation;
- unit of observation: question, image, slide, case, or patient;
- official split and any regrouping;
- image resolution, format, duplicates, and missing files;
- question and answer distributions;
- closed/open proportions and answer vocabulary coverage;
- provenance, demographic information if available, and known gaps;
- intended research use and prohibited interpretation.

19.4 Leakage and duplication

Several questions can refer to the same image. Splitting question rows can therefore place the same image in train and validation. Audit image IDs and perceptual hashes, as well as near-duplicate question templates. If patient, slide, or case identifiers exist, group at the highest relevant dependency level.

19.5 Evaluation protocol

Report at minimum:

- total sample counts and answer-type counts;
- closed yes/no accuracy, balanced accuracy, and confusion matrix;
- open-ended Exact Match and token F1 under declared normalization;
- overall results only alongside the stratified results;
- majority-answer and question-only baselines;
- variation across seeds or an explicit uncertainty limitation.

For yes/no prevalence p, majority accuracy is $\max(p, 1-p)$. A multimodal model that does not exceed question-only controls under the same evaluation has not demonstrated useful visual contribution.

19.6 Notebook audit

Review the supplied `Lab_Week6_PathVQA_GPT_LoRA.ipynb` [28] and record only:

1. data flow and split assumptions;
2. visual and language model interfaces;
3. projector or fusion mechanism;
4. LoRA targets, rank, alpha, and trainable parameters;

5. loss, decoding, and metric definitions;
6. hardware assumptions and missing reproducibility details.

Implement independently rather than copying long notebook sections.

19.7 Failure taxonomy

For the core route, categorize a small supplied or synthetic set of errors. Reviewing at least 20 real-model errors is an Optional Heavy extension. Use provisional causes such as:

- **visual:** evidence is small, cropped, low-resolution, or missed;
- **language:** question parsing, negation, or template confusion;
- **knowledge:** answer may require information not visible in the image;
- **fusion:** one modality appears ignored;
- **decoding/metric:** output formatting or normalization mismatch;
- **annotation:** ambiguous, incomplete, or questionable reference.

These are analysis labels, not clinical adjudications.

19.8 Medical-use limitations

Every report must state that the experiment is not clinically validated; dataset performance does not establish diagnostic safety; subgroup and external-site generalization may be unknown; errors may be confidently phrased; data provenance and label quality constrain conclusions; and human expert review would be required for any clinical research pathway.

19.9 Hands-on work

Use `exercises/day19_pathvqa.py`.

1. Build the data card and split-overlap report.
2. On the included synthetic fixture, compute the majority baseline and specify how question-only and image-plus-question baselines would be trained.
3. Define separate closed and open-ended result tables.
4. Specify both modality-shuffle ablations and their expected interpretation.
5. Categorize the supplied failures; real-data training and a 20-case review are Optional Heavy.

19.10 Acceptance questions

1. Why are majority and question-only baselines essential?
2. Why can question-row splitting leak images?
3. Which limitations prevent clinical interpretation?
4. What does image-shuffle degradation establish and not establish?

5. Why must closed and open-ended metrics be separated?

Chapter 20

Capstone, Reproduction Package, and Oral Defense

Study level: Core Small / Optional Heavy

Complete a lightweight or synthetic capstone design. Real PathVQA training and large-model adaptation are optional follow-on work.

Learning objectives

- run a controlled end-to-end VQA comparison;
- package data, model, metrics, and environment information reproducibly;
- separate observations from causal conclusions;
- present failure analysis and limitations;
- defend the full 100-hour learning path orally.

20.1 Five-hour plan

25 min planning; 25 min equation review; 180 min experiments; 20 min source check; 50 min defense.

20.2 Smallest runnable check

Run `exercises/day20_capstone.py` first. It verifies that the lightweight image–question classifier can overfit a deliberately tiny synthetic batch. Passing this check is necessary but not sufficient for the capstone comparison below.

20.3 Minimum experiment

Use one fixed, legally obtained dataset version and compare:

1. majority-answer baseline;
2. question-only baseline;

3. lightweight image-plus-question fusion;
4. optional frozen visual/language base plus LoRA.

Use identical splits, preprocessing, and evaluation code. Without real PathVQA data, label the work *PathVQA-style synthetic experiment*; do not present it as a conclusion about PathVQA.

20.4 Pre-registered questions

Before running, specify the primary metric, comparison, success threshold, and selection rule. Example: image-plus-question fusion must exceed the question-only mean by more than observed seed variation while passing image-shuffle sensitivity.

20.5 Required result table

Method	Params	Trainable	Closed acc.	Open EM	Open F1	Peak GiB
Majority						
Question-only						
Image+question						
Optional LoRA						

Also report runtime, sample counts, checkpoint-selection step, and modality-shuffle results. Report seed variation when repeated runs are feasible; otherwise mark it as an unestimated limitation. Compare methods only where evaluation populations match.

20.6 Reproduction package

Include:

- environment lock file and hardware description;
- seed policy and deterministic settings;
- data source, version, license, checksums, and split IDs or script;
- model and tokenizer revisions;
- full configuration and trainable-parameter list;
- checkpoint-selection rule and exact evaluation command;
- raw and summarized metrics, logs, and negative results;
- a small set of de-identified or synthetic failure records; expand to at least 20 for Optional Heavy real-data work;
- limitations, intended use, and non-clinical-use statement.

Run the repository smoke tests and unit tests before submission. A README should reproduce the smallest path first, then describe the optional heavyweight route.

20.7 Evidence ladder

Use precise claims:

- one successful run is an *observation*;
- repeated controlled runs support a *comparative result*;
- an ablation can support a mechanism hypothesis but rarely proves causation;
- one dataset does not establish clinical generalization;
- benchmark performance does not establish safety or utility.

20.8 Ten-minute defense

Present in this order:

1. problem and non-clinical scope;
2. data provenance, split, and leakage audit;
3. baselines and model architecture;
4. objective and metrics;
5. results with uncertainty;
6. modality-use tests and failure cases;
7. limitations and next experiment.

Be ready to derive attention shapes, LoRA parameter counts, QLoRA memory accounting, ViT token growth, CLIP loss, and cross-attention complexity without notes.

20.9 Final acceptance

1. Can another person reconstruct the split and evaluation?
2. Do multimodal models beat question-only controls under the same protocol?
3. Does image shuffling cause the expected degradation?
4. If repeated runs were feasible, are improvements larger than seed variation; otherwise is uncertainty stated explicitly?
5. Are negative results and failed runs preserved?
6. Are all external methods and notebooks cited?
7. Does the report avoid clinical claims and identify deployment gaps?

Part V

Reference Appendices

Appendix A

Mathematical Reference

A.1 Shape convention

Throughout the book, B denotes batch size, T sequence length, C feature count, O output feature count, and K the number of classes. A matrix multiplication

$$\boldsymbol{X}_{B\times C}\boldsymbol{W}_{C\times O} = \boldsymbol{Y}_{B\times O}$$

is valid because the inner dimensions match.

A.2 Frequently used derivatives

For vectors and matrices of compatible shape:

$$\begin{aligned}
\nabla_{\boldsymbol{x}}(\boldsymbol{a}^\top \boldsymbol{x}) &= \boldsymbol{a}, \\
\nabla_{\boldsymbol{x}} \frac{1}{2}\|\boldsymbol{x}\|_2^2 &= \boldsymbol{x}, \\
\nabla_{\boldsymbol{w}} \frac{1}{B}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2 &= \frac{2}{B}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}), \\
\frac{d}{dx}\tanh x &= 1 - \tanh^2 x.
\end{aligned}$$

A.3 Probability reminders

For class logits $z_1, \dots, z_K$,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \mathcal{L} = -\log p_y.$$

The stable implementation subtracts $\max_j z_j$ before exponentiation. This changes neither the probabilities nor the loss.

A.4 Convolution reminders

For a one-dimensional spatial axis with input size H, kernel size K, padding P, stride S, and dilation D, the output size is

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} + 1 \right\rfloor.$$

A standard two-dimensional convolution with C_{in} input channels, C_{out} output channels, and a $K_h \times K_w$ kernel has

$$C_{\text{out}} C_{\text{in}} K_h K_w + C_{\text{out}}$$

parameters when a bias is used.

A.5 Recurrent-network reminders

A vanilla recurrent state can be written as

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h).$$

Backpropagation through time contains products of recurrent Jacobians,

$$\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \boldsymbol{h}_j}{\partial \boldsymbol{h}_{j-1}},$$

which helps explain vanishing or exploding gradients. LSTM and GRU gates create controlled paths for retaining, updating, and exposing state; they reduce but do not eliminate every long-range optimization problem.

Appendix B

Week 1 Foundations Checklist

Before claiming that a Week 1 experiment is complete, verify the following.

- ☐ The random seed and data split are recorded.
- ☐ Tensor shapes and dtypes are asserted at important interfaces.
- ☐ Preprocessing statistics are fit on training data only.
- ☐ Training and validation use the correct model mode.
- ☐ Validation runs without gradient tracking.
- ☐ Metrics are aggregated by sample, not by unweighted batch average.
- ☐ A one-batch overfit test succeeds.
- ☐ I can derive both weight gradients of a two-layer MLP and verify one element by finite differences.
- ☐ Optimizer, learning rate, weight decay, update count, and scheduler are recorded.
- ☐ I can compute a convolution output shape and parameter count before running code.
- ☐ A tiny CNN can overfit a deliberately small image batch.
- ☐ I can trace an RNN hidden state and explain the purpose of LSTM or GRU gates.
- ☐ I can state why recurrence has a sequential bottleneck and how self-attention differs.
- ☐ The checkpoint contains model, optimizer, step, and metadata.
- ☐ A fresh process can load the checkpoint and reproduce predictions.
- ☐ Failed runs and unexpected behavior are recorded rather than deleted.

Recommended Codex review prompt

> Review this experiment for tensor-shape errors, data leakage, incorrect train/eval modes, device or dtype mismatches, random-seed omissions, and metric aggregation bugs. List findings before editing any file.

Appendix C

Attention Shape and Parameter Reference

C.1 Core symbols

Symbol	Meaning	Typical tensor role
B	batch size	independent sequences
T	sequence length	token positions
V	vocabulary size	output classes
C	model width	residual-stream channels
H	attention heads	parallel matching subspaces
D	head width, C/H	per-head channels
L	number of blocks	model depth
M	feed-forward hidden width	intermediate channels

C.2 End-to-end shape trace

Operation	Output shape	Check
Token IDs	(B, T)	integer IDs in $[0, V)$
Token embedding	(B, T, C)	lookup from (V, C)
Combined QKV projection	$(B, T, 3C)$	one projection, then split
Split Q, K, V	three (B, T, C) tensors	final axis split equally
Split heads	(B, H, T, D)	$D = C/H$
Attention scores	(B, H, T, T)	last two axes are query/key positions
Attention output by head	(B, H, T, D)	weights times values
Merge heads	(B, T, C)	transpose, contiguous, reshape
Vocabulary projection	(B, T, V)	one distribution per position
Flatten for loss	(BT, V)	targets become (BT)

C.3 Approximate parameter counts

Ignoring biases and normalization parameters:

$$\begin{aligned}
\text{token embedding} &: VC,\\
\text{learned positions} &: T_{\max}C,\\
\text{attention projections per block} &: 4C^2,\\
\text{two-matrix MLP with ratio } r &: 2rC^2,\\
\text{SwiGLU with hidden width } M &: 3CM,\\
\text{untied output head} &: VC.
\end{aligned}$$

C.4 Mask sanity checks

For a causal attention matrix $A \in \mathbb{R}^{T\times T}$:

- $A_{ij} = 0$ whenever $j > i$;
- every row sums to one;
- row 0 has exactly one allowed position;
- changing tokens after position i cannot change output positions $\leq i$.

Appendix D

Week 2 Acceptance Checklist

Mark an item only when you can reproduce it without copying the notes.

Conceptual

- ☐ I can explain encode/decode, vocabulary, special tokens, and shifted targets.
- ☐ I can distinguish a causal mask from a padding mask.
- ☐ I can explain query-key matching and value aggregation.
- ☐ I can distinguish token mixing from channel mixing.
- ☐ I can explain teacher-forced training and autoregressive generation.
- ☐ I can compare LayerNorm, RMSNorm, learned positions, and RoPE.

Mathematical

- ☐ I can derive sequence negative log-likelihood and perplexity.
- ☐ I can derive the $1/\sqrt{D}$ attention scale from a variance argument.
- ☐ I can trace all attention shapes from (B, T, C) back to (B, T, C).
- ☐ I can derive the residual Jacobian $I + J_F$.
- ☐ I can derive the relative-displacement identity for RoPE.
- ☐ I can estimate block and embedding parameter counts by hand.

Implementation

- ☐ My input and target windows pass the one-position shift assertion.
- ☐ My attention passes row-sum, triangular-mask, and future-invariance tests.
- ☐ Padded keys receive zero attention and padded targets do not contribute to the loss.
- ☐ My Transformer block preserves shape and can become an identity map.
- ☐ My Tiny GPT can overfit one fixed batch.

- ☐ My sampling code supports deterministic greedy decoding and seeded sampling.
- ☐ My RMSNorm, RoPE, and SwiGLU pass component-level tests.

Weekly evidence

Keep these artifacts in the experiment log:

1. a handwritten or digital attention shape trace;
2. a screenshot or table of the future-invariance test;
3. a Tiny GPT loss curve showing fixed-batch overfitting;
4. samples under at least three decoding settings;
5. a GPT-versus-LLaMA component comparison;
6. one completed paper-reading template.

Checkpoint

Week 2 is complete only when you can explain the architecture and demonstrate the tests. A generated text sample by itself is not evidence that the implementation is correct.

Appendix E

LoRA and QLoRA Reference

E.1 LoRA shapes

For $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

Object	Shape	Status
W_0	$(d_{\text{out}}, d_{\text{in}})$	frozen base
A	(r, d_{in})	trainable adapter
B	(d_{out}, r)	trainable adapter
BA	$(d_{\text{out}}, d_{\text{in}})$	low-rank update
input x	$(\ldots, d_{\text{in}})$	activation
output y	$(\ldots, d_{\text{out}})$	activation

E.2 Quick formulas

$$\Delta W = (\alpha/r)BA,$$
$$N_{\text{adapter}} = r(d_{\text{in}} + d_{\text{out}}),$$
$$M_{\text{ideal}} = Nb/8 \text{ bytes},$$
$$1 \text{ GiB} = 2^{30} \text{ bytes},$$
$$1 \text{ GB} = 10^{9} \text{ bytes}.$$

E.3 Configuration manifest

Field	Value
Base model ID and revision	
Tokenizer ID and revision	
License/access condition	
Task formulation	
Target modules	
Rank, alpha, dropout	

Field	Value
Quantization type and block settings	
Compute dtype	
Sequence length and batch settings	
Optimizer and scheduler	
Dataset version and split rule	
Normalization and metric version	
Hardware and software versions	
Measured peak memory definition	

E.4 Terminology check

- **Quantized:** represented with a low-bit code plus metadata.
- **Dequantized:** reconstructed into a compute representation for an operation.
- **Frozen:** excluded from parameter updates.
- **Trainable:** receives gradients and optimizer updates.
- **Merged:** adapter update added into a base weight for inference.

Appendix F

Week 3 Acceptance Checklist

LoRA

- ☐ I can derive adapter shapes and parameter counts.
- ☐ I can explain why the update, not the base, is low rank.
- ☐ Initial equivalence and frozen-gradient tests pass.
- ☐ Merged and unmerged outputs agree within tolerance.
- ☐ I recorded target modules, rank, alpha, and dropout.

QLoRA and memory

- ☐ I can state exactly what is quantized, frozen, and trained.
- ☐ I can distinguish storage dtype from compute dtype.
- ☐ I can explain NF4, double quantization, and paged optimizers.
- ☐ My ledger separates weight floor from measured peak memory.
- ☐ My memory report includes hardware and complete batch settings.
- ☐ I understand that actual 4-bit large-model fine-tuning is Optional Heavy, not a first-month requirement.

QA and engineering

- ☐ I inspected target modules instead of guessing their names.
- ☐ I saved the adapter, tokenizer, base revision, and configuration.
- ☐ EM, F1, normalization, and empty-answer tests pass.
- ☐ I audited question, image, and patient/study overlap.
- ☐ I reported metrics with counts and meaningful strata.

Rank ablation

- ☐ Hypotheses and the rank-selection rule were written in advance.
- ☐ At least two ranks received a fair lightweight budget and identical evaluation rules.
- ☐ Results include seed variation or explicitly state that it was not estimated.
- ☐ Parameter, metric, runtime, and memory results are recorded; multi-seed language-model experiments are Optional Heavy.
- ☐ Negative and inconclusive runs were retained and discussed.

Checkpoint

Week 3 is complete when the adapter mechanics pass tests and a small controlled comparison supports a cautious conclusion. A successful library call alone is not evidence of understanding; full QLoRA training and multi-seed language-model ablations remain optional follow-on work.

Appendix G

Multimodal Shape and Evaluation Reference

G.1 Shape table

Object	Shape	Meaning
Images	(B, C_{in}, H, W)	input pixels
ViT patches	(B, N, C_v)	$N = (H/P)(W/P)$ visual tokens
Question tokens	(B, T, C_l)	language representations
Projected visual tokens	(B, N, C_l)	visual tokens in language width
Cross-attention scores	(B, H, T, N)	text queries by visual keys
Closed-answer logits	(B, K)	fixed answer vocabulary
Generative logits	(B, T_a, V)	answer-token distributions
CLIP similarity	(B, B)	paired batch similarities

G.2 Core formulas

$$
\begin{aligned}
N &= (H/P)(W/P), \\
\text{ViT score elements per head} &= N^2, \\
S_{ij}^{\text{CLIP}} &= u_i^\top v_j / \tau, \\
\text{cross-attention score shape} &= (B, H, T, N), \\
\text{linear projector parameters} &= C_l C_v + C_l.
\end{aligned}
$$

G.3 Minimum baseline ladder

1. majority answer;
2. question only;
3. image only;
4. pooled image plus question;
5. proposed multimodal model;

6. image-shuffled and question-shuffled ablations.

Appendix H

Week 4 and Final Acceptance Checklist

Vision and contrastive learning

- ☐ I can trace ViT patch shapes and calculate attention growth.
- ☐ I recorded image resolution, patch size, and preprocessing.
- ☐ I can derive the symmetric CLIP loss.
- ☐ I report retrieval in both directions and handle duplicates explicitly.

VLM and PathVQA

- ☐ I can compare pooled fusion, projection, and cross-attention.
- ☐ Majority, question-only, and image-only controls are included.
- ☐ Image and question shuffle tests are saved.
- ☐ Closed and open-ended metrics are reported separately.
- ☐ Image, template, and group leakage audits are documented.
- ☐ Failure cases are categorized; 20 or more real-data cases are an Optional Heavy extension.

Reproducibility and boundaries

- ☐ Data license, version, split, model revision, and environment are recorded.
- ☐ Raw runs, seeds, negative results, and checkpoint rules are retained.
- ☐ External papers, code, and notebooks are cited.
- ☐ The report states that the experiment is not clinically validated.
- ☐ Synthetic experiments are labeled as synthetic.

Checkpoint

The 100-hour program is complete when you can reproduce a small local or synthetic experiment, defend its mathematics, identify what each modality contributes, and state precisely what the evidence does not establish. Real PathVQA training is optional follow-on work.

Bibliography

Bibliography

[1] Kie Codes. *Code Your First Neural Network with PyTorch*. YouTube video and MIT-licensed repository, 2021. https://github.com/kiecodes/first-nn-pytorch.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. https://www.deeplearningbook.org/.

[3] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.

[4] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.

[5] Xavier Glorot and Yoshua Bengio. Understanding the Difficulty of Training Deep Feedforward Neural Networks. *AISTATS*, 2010.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving Deep into Rectifiers. *ICCV*, 2015.

[7] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 1998.

[8] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 1997.

[9] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *EMNLP*, 2014.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. *Attention Is All You Need*. NeurIPS, 2017.

[11] Alec Radford, Jeffrey Wu, Rewon Child, et al. *Language Models are Unsupervised Multitask Learners*. OpenAI technical report, 2019.

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. *Language Models are Few-Shot Learners*. NeurIPS, 2020.

[13] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.

[14] Biao Zhang and Rico Sennrich. *Root Mean Square Layer Normalization*. NeurIPS, 2019.

[15] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, 2021.

[16] Noam Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.

[17] PyTorch Contributors. *PyTorch Documentation*. https://pytorch.org/docs/stable/, accessed 2026.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021.

[19] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv:2305.14314, 2023.

[20] Hugging Face. *PEFT Configurations and Models*. https://huggingface.co/docs/peft/guides/peft_model_config, accessed 2026.

[21] Hugging Face. *PEFT LoRA API Reference*. https://huggingface.co/docs/peft/package_reference/lora, accessed 2026.

[22] Hugging Face. *PEFT Model API Reference*. https://huggingface.co/docs/peft/package_reference/peft_model, accessed 2026.

[23] Hugging Face. *Transformers Bitsandbytes Quantization Guide*. https://huggingface.co/docs/transformers/quantization/bitsandbytes, accessed 2026.

[24] Mobarakol Islam. *LLM GPT-2, LLaMA, QA, and LoRA Tutorial Notebook*. https://github.com/mobarakol/tutorial_notebooks/blob/main/LLM_GPT2_LLaMA_QA_LoRA_FINAL.ipynb, accessed 2026.

[25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929, 2020.

[26] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML, 2021.

[27] Xuehai He, Zhuo Cai, Wenlan Wei, et al. *Towards Visual Question Answering on Pathology Images*. ACL-IJCNLP Short Papers, 2021.

[28] Mobarakol Islam. *Lab Week 6: PathVQA, GPT, and LoRA Notebook*. https://github.com/mobarakol/Computational_Techniques_for_Multimodal_Data/blob/main/Lab_Week6_PathVQA_GPT_LoRA.ipynb, accessed 2026.